Proxmox Storage DRS

Balancing disk I/O across shared storages on Proxmox VE 9.2

Bernd Zeimetz bernd@bzed.de

2026-09-08

Rendered from `IMPLEMENTATION_PLAN.md`, SHA-256 `62c4ce22c490d284212b3bcfeb256d3f08133cd7d4ef84fc78c907180d662755`
(`sha256sum --check docs/IMPLEMENTATION_PLAN.pdf.sha256` in the repository must succeed).

Contents

A specification for a VMware Storage DRS replacement targeting **Proxmox VE 9.2**. It is written to be implementable without further research: every external fact it relies on is stated inline, and section 14 is a worked numeric example that doubles as a test fixture.

---

1. Scope

Goal

Given configurable **groups** of shared storages, continuously equalize **disk I/O load** across the storages within each group by live-migrating individual VM disks, subject to:

- a disk may only move between storages in **its own group**;
- every storage must always retain free space for snapshots — by default **2× its largest disk** — and this must hold *during* migrations, not merely before and after;
- the number of migrations must be **minimal**;
- a VM's disks should stay **together** on one storage unless space or I/O forces otherwise;
- a migration's own I/O cost must not exceed the imbalance it removes.

Non-goals

Compute/CPU/memory DRS, VM-to-node placement, HA, backup scheduling, storage provisioning, thin-pool overcommit management, and any modification of guest configuration beyond disk location.

Relationship to the PVE 9.2 Dynamic Load Balancer

PVE 9.2 ships a built-in Dynamic Load Balancer — a dynamic mode for the Cluster Resource Scheduler that continuously live-migrates **HA-managed guests between nodes** to even out node CPU/memory utilization. It is **complementary, not overlapping**: it balances guests across *hypervisors* and has no notion of storage or of the FC LUNs behind it. Nothing in PVE balances disk I/O across storages, which is the gap this tool fills.

There is, however, a real interaction: the built-in balancer may live-migrate a VM to another node *while a Storage DRS plan is executing*, invalidating the `{node}` in a queued `move_disk` call. The pre-move re-validation in §9.2 exists precisely to catch this — it re-reads the VM's current node before every move and re-plans on mismatch. Implementers must not cache the node across moves.

Requirement interpretation: "minimal number of migrations"

The requirement that migrations be minimal is implemented as a **tunable preference** (the β term in §5.4), not as a strict lexicographic minimum. A strict minimum would refuse a second cheap move that halves the remaining imbalance, which is not what is wanted. β sets the exchange rate between "one more migration" and "this much less imbalance"; §14.3 demonstrates β selecting a two-move plan over a three-move plan on the same input.

Operating assumptions

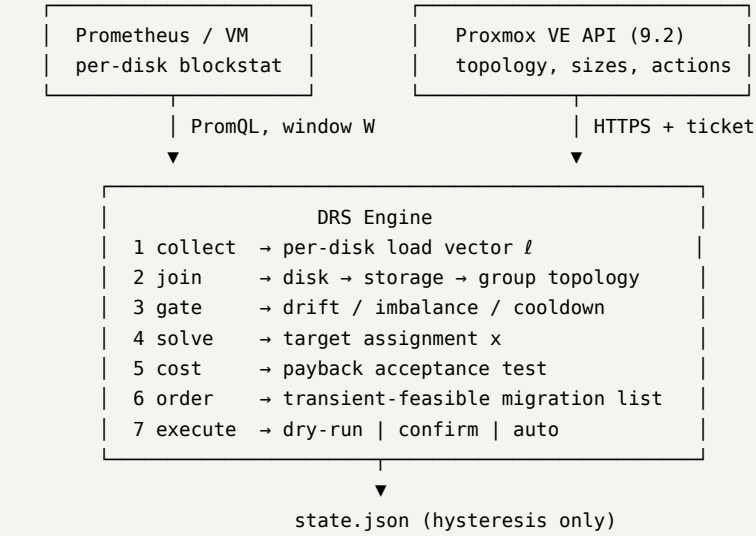
- PVE 9.2, shared LVM (typically over FC) or any shared storage supporting `move_disk`.
- Only **running** VMs are considered by default; a stopped VM generates no I/O to balance, and its disks are moved only if a capacity constraint requires it.
- Disks are thick-provisioned by default, so migration cost is proportional to *provisioned* size.
- **A VM is never stopped, suspended or reconfigured.** The only write the engine issues is `move_disk`. This is what rules out the offline path for snapshotted disks (§3.7), which are movable only with the guest down — a maintenance-window decision for a human.
- Every bus is in scope and every one of them moves online: `ide`, `sata`, `scsi`, `virtio`, plus `efidisk0`, `tpmstate0` and `unused` volumes (§3.6). A VM's disks are not all `scsi*`, and anything not enumerated is capacity the model cannot see (§3.5).

- Storage-side cleanup is part of a migration, not an afterthought: with LVM `saferemove` the source volume is zeroed at a throttled rate after the mirror completes, and neither its space nor the VM is available until that finishes (§7.1, §9.3).

---

2. Architecture

The engine is **stateless with respect to metrics** — all history lives in Prometheus. It keeps one small local state file purely for hysteresis.



Two data sources are genuinely required, and neither substitutes for the other:

- **Prometheus** knows the load of `vmid=101, device=scsi0` but has **no idea which storage that disk lives on**.
- **The PVE API** knows the topology, sizes and capacities, and is the only way to *act*.

The join between them is the disk identity `(vmid, device)`, which both sides expose.

Module layout

Module	Responsibility
`config.py`	Load/validate YAML, defaults, unit parsing (`24h`, `200MiB`), `/…/` storage patterns (§11.4)
`metrics.py`	Prometheus client, PromQL construction, `verify-metrics`
`pve.py`	API client: auth, topology, storage status, `move_disk`, task polling
`topology.py`	Build the disk/storage/group model, eligibility rules, storage-pattern expansion (§11.4)
`loadmodel.py`	Reduce raw series to the scalar load vector ℓ
`forecast.py`	Pluggable forecaster (quantile, seasonal-naive, Holt-Winters)
`optimize.py`	MILP formulation (CP-SAT / CBC)
`heuristic.py`	Dependency-free greedy + local search fallback
`payback.py`	Migration cost model and acceptance test
`schedule.py`	Ordering under the transient reserve invariant
`execute.py`	Three execution modes, task supervision
`cli.py`	`plan`, `apply`, `verify-metrics`, `verify-storages`, `show-load`, `explain`

The installed executable is **`pve-storage-drs`** — one `[project.scripts]` entry point onto `cli.py`. Every

command in this document is written as `pve-storage-drs <subcommand>`; the manpage is `pve-storage-drs(1)`, and the Debian source and binary package are `pve-storage-drs` too. The Python package keeps its distribution name `proxmox-storage-drs` and its import name `proxmox_storage_drs`. The word **storage** is load-bearing: PVE 9.2's own Dynamic Load Balancer moves guests between nodes (§1), and a shorter name would suggest this tool replaces it.

2.1 Implementation, deployment and operations

Language: Python 3.11+. The decision is driven by the solver and forecasting libraries — OR-Tools CP-SAT and `statsmodels` have no usable equivalent in Go or Rust without substantial reimplementation, and PVE hosts already ship Python, so operators can read and patch the tool. Single-binary deployment is not a requirement here; if it ever becomes one, the dependency-free heuristic path (§5.5) is the portable subset worth porting.

Dependencies come from Debian. The deployment target runs Proxmox VE 9.x, which is built on Debian trixie, so a module that trixie packages is a module the operator already trusts, already patches through their normal update path, and already has on a host with no outbound network. Every dependency below is therefore chosen for being in trixie, and the tool is packaged as a `.deb` (§2.2). A module that is *not* in Debian is only acceptable if it is pure Python and small enough to vendor into our own source package; anything else must be optional, with a code path that works without it.

Package	Purpose	In Debian trixie
`requests`	Prometheus HTTP, and the transport `proxmoxer`'s https backend uses	`python3-requests` 2.32
`proxmoxer`	PVE API client (§3.5)	`python3-proxmoxer` 2.2
`ruamel.yaml`	Config (round-trips comments)	`python3-ruamel.yaml` 0.18
`jsonschema`	Config validation (§11.1)	`python3-jsonschema` 4.19
`pulp`	MILP via CBC — **the packaged solver path**, optional	`python3-pulp` 2.7 + `coinor-cbc` 2.10
`statsmodels`	Holt-Winters, optional	`python3-statsmodels` 0.14
`ortools`	CP-SAT, optional and unpackaged	**not in Debian**
`pytest`, `pytest-cov`, `pytest-xdist`	Tests; groups are independent so they parallelize	`python3-pytest*`

`ortools` is the one dependency Debian does not carry, and it is not a candidate for vendoring: it is a large C++ extension, not a pure-Python module. It therefore stays a **pip-only optional extra**, and CP-SAT is a bonus for whoever installs it rather than the assumed backend. §5.5 must be read accordingly: on a Debian install the MILP is solved by **CBC through `python3-pulp`**, with the dependency-free heuristic below that.

Make `ortools`, `pulp` and `statsmodels` **optional extras**. The tool must run, plan and execute with only `requests` + `ruamel.yaml` + `jsonschema` installed, falling back to the heuristic solver and the quantile forecaster. `pulp` is the odd one: it is *in* Debian and it is the solver an operator should have, so it is `Recommends` in `debian/control` and an extra rather than a dependency in `pyproject.toml` — "the packaged solver path" describes which MILP backend a Debian install gets, not that the MILP is mandatory. Without any solver the tool still plans, using the heuristic of §5.5; that is the whole point of specifying two of them. This keeps it deployable on a locked-down management host — and it is enforced rather than hoped for: the autopkgtest in §2.2 imports every module of the installed package with only the binary package's `Depends` present, so an optional dependency imported at module level fails the build.

Licence and contribution rules. The project is **AGPL-3.0-or-later**, copyright Bernd Zeimetz bernd@bzed.de; every source file carries the two-line SPDX header. `AGENTS.md` and `.agents/` hold the working agreement that any implementer — human or model — is expected to follow: black + isort + flake8 + mypy

with configurations written so the formatter and the linter cannot disagree, an enforced 85% coverage floor, feature branches merged only when `make check` is green, and the list of domain invariants (dry-run default, reserve never traded, no auto-delete) that a refactor must not quietly remove. A dependency whose licence is incompatible with AGPL-3.0-or-later cannot be added.

Deployment. Runs on a management host — not necessarily a PVE node — needing outbound access to the PVE API (tcp/8006) and to Prometheus. It holds credentials and must be treated accordingly; prefer an API token over username/password for unattended operation.

Cadence. Invoked by a systemd timer (or cron) every 15–30 min. Note the separation of concerns: the *invocation* cadence is independent of `execution.time_windows`, which constrain only when **moves execute**. The engine may plan at any time and simply decline to act outside the window. The drift and imbalance gates (§6) make frequent invocation cheap — most runs exit at a gate having issued only read queries.

Logging. Structured JSON lines to **stderr** (captured by journald) plus an optional file sink. Logging goes to stderr rather than stdout specifically so that `--json`'s machine-readable plan report (section 9.5) can be safely captured from stdout alone; a systemd service unit captures both streams into the same journal, so nothing is lost when run under the timer. Every run must log, at minimum: each gate decision with its computed value and threshold; the load vector digest; the chosen plan and its objective breakdown; the payback arithmetic; every `move_disk` issued with its UPID; and every abort, re-plan and deadlock. In `auto` mode this log is the only record a human will see, so it must be sufficient to reconstruct why any migration happened.

2.2 Packaging and continuous integration

The tool is delivered as a Debian package, `pve-storage-drs`, built from `debian/` in this repository (source format `3.0 (native)`, since upstream and packaging are the same tree). It installs the executable, the manpage, the example configuration and the generated documentation. Two rules follow from that and are not negotiable:

- **`debian/control` is the single source of truth for dependencies.** Build-Depends and Depends are updated in the same commit as the code or documentation that needs them. CI installs the build dependencies *from* `debian/control` with `mk-build-deps`, so a stale declaration fails the build rather than working by accident on a developer's machine.
- **The autopkgtest asks the only question the build cannot.** The build chroot has the Build-Depends installed and therefore cannot notice a missing runtime dependency. `debian/tests` installs the built package on a system that has only its `Depends`, then runs `pve-storage-drs --version`, `pve-storage-drs --help`, and an import of every module in the package.

Two pipelines, deliberately different:

	Runs	Covers
GitHub Actions	`debian:trixie` containers	lint, types, tests and coverage with Debian's packaged tooling; the document build against trixie's older pandoc; `dpkg-buildpackage`, lintian, and install-then-run
GitLab CI (Salsa)	Debian's Salsa CI pipeline	sbuild in an unshare chroot with **no network**, then lintian, piuparts, reprotest and autopkgtest

The Salsa build having no network is the load-bearing part: it is what proves the package builds from trixie alone. If a Python module ever has to be fetched during a build, the answer is to vendor it into the

source package; enabling `--enable-network` for sbuild is the fallback, and it is a deliberate, visible change to `debian/.gitlab-ci.yml`, not a default.

---

3. Data acquisition

3.1 Where the per-disk data comes from

This is the single most important implementation fact, so it is stated precisely.

`pvestatd` calls `PVE::QemuServer::vmstatus(undef, 1)` — note the `full=1` flag. With `full` set, `vmstatus` issues QMP `query-blockstats` and stores the result as:

```
$res->{$vmid}->{blockstat}->{$drive_id} = $blockstat->{stats};
```

where `$drive_id` is the QMP device name with the `drive-` prefix stripped — i.e. **`scsi0`, `virtio0`, `ide2`**, matching the VM config keys exactly. The whole QMP stats hash is kept without filtering, which includes:

```
rd_bytes  wr_bytes  rd_operations  wr_operations
rd_total_time_ns  wr_total_time_ns
flush_operations  flush_total_time_ns
rd_merged  wr_merged  failed_rd_operations  failed_wr_operations
invalid_rd_operations  invalid_wr_operations  account_invalid  account_failed  idle_time_ns
```

`PVE/Status/InfluxDB.pm` then flattens the nested hash so that the **first** nesting level becomes the InfluxDB *measurement* (`blockstat`), the **second** becomes an *instance tag* (`instance=scsi0`), and the leaves become *fields*. Combined with the base tags from `update_qemu_status` (`object=qemu`, `vmid=…`, `nodename=…`, `host=<vm name>`), a series arrives in Prometheus roughly as:

```
blockstat_rd_operations{vmid="101", instance="scsi0", nodename="pve01", host="db-01"}
```

Everything needed for true per-disk IOPS, throughput **and latency** is therefore already present in an existing PVE → InfluxDB → Telegraf → Prometheus pipeline. **No new exporter or collector is required.**

Verification status of the above. The `vmstatus(undef, 1)` call, the `blockstat->{$drive_id}` assignment, the `s/drive-//r` prefix stripping and the `InfluxDB.pm` nesting behaviour were read from the `pve-manager` and `qemu-server` sources, not inferred from documentation. Independently, the operator of the target cluster confirmed empirically that their existing Prometheus already carries `rd_operations`, `wr_operations`, `rd_bytes`, `wr_bytes`, `rd_total_time_ns` and `wr_total_time_ns` per disk. The *existence* of the data is therefore settled.

What remains genuinely unconfirmed is the **Telegraf-side naming** — the measurement/field join character, and whether the `instance` tag survived the collision described in §3.3. That varies per deployment and is exactly what `pve-storage-drs verify-metrics` exists to pin down. Treat §3.4's metric names as defaults to be confirmed, not as constants.

3.2 Two paths deliberately not taken

Do not use RRD. `GET /nodes/{node}/qemu/{vmid}/rrddata` exposes only VM-aggregate `diskread`/`diskwrite` in bytes/s — no per-disk breakdown and no operation counts. `GET /nodes/{node}/storage/{storage}/rrddata` exposes only `used` and `total`, with no I/O metrics at all. Prometheus supersedes RRD entirely here.

Do not switch to the OpenTelemetry metric server (new in PVE 9.1), despite it being the more modern-looking option. Its `_convert_node_metrics_recursive` handles a nested hash by appending the key to the metric **name**:

```
if (ref($value) eq 'HASH') {
    push @metrics, $class->_convert_node_metrics_recursive(
        $value, $ctime, "${metric_prefix}_${key}", $attributes,   # <-- name, not attribute
    );
}
```

so `blockstat.scsi0.rd_operations` becomes the metric **`proxmox_vm_blockstat_scsi0_rd_operations_total`**. The device is baked into the metric name rather than carried as a label, which makes PromQL aggregation across devices impossible without `label_replace` gymnastics and produces unbounded metric-name cardinality. The InfluxDB path puts the device in a **label** and is strictly superior for this use case. (The plugin also only supports OTLP/JSON encoding, not protobuf.)

3.3 Metric name mapping and verification

Telegraf naming is deployment-specific, so **every metric and label name is configuration**, not a constant. Implement `pve-storage-drs verify-metrics` as a first-class command that must be run before anything else. It shall:

1. query `/api/v1/label/__name__/values` and confirm each configured metric name exists;
2. run one instant query per metric and print a sample series with all labels;
3. confirm the configured `vmid`, `device` and `node` labels are present and non-empty;
4. warn loudly if the device label is literally `instance` — **PVE's `instance` tag collides with Prometheus's own scrape-target `instance` label**, and many Telegraf configurations rename or overwrite it. The implementer must confirm the real label name here before proceeding;
5. report per-disk sample coverage over the configured window, so gaps are visible up front;
6. measure the **observed sample spacing** of a live series (the modal delta between consecutive timestamps in a short `query_range`) and compare it against `metrics.pvestatd_push_interval`. That interval is a PVE-side setting the tool cannot read from the API, so it is declared in config; this step is what stops a stale declaration from silently invalidating the `rate_window ≥ 4 × interval` rule of §11.1. Error if the two disagree by more than 20%, and error if `rate_window` is below four times the *observed* spacing regardless of what config claims.

3.4 PromQL

Let `R` be `metrics.rate_window` (default `5m`) and `W` be `window.lookback` (default `24h`). Three raw quantities per disk, all keyed by `(vmid, device)`:

Read and write are fetched **separately** so that `read_factor`/`write_factor` (§4) can be applied engine-side — six queries per group, not three:

```
# in-flight I/O, read and write (divide by 1e9 to get s/s)
sum by (vmid, device) (rate(blockstat_rd_total_time_ns[5m])) / 1e9
sum by (vmid, device) (rate(blockstat_wr_total_time_ns[5m])) / 1e9

# operations per second
sum by (vmid, device) (rate(blockstat_rd_operations[5m]))
sum by (vmid, device) (rate(blockstat_wr_operations[5m]))

# bytes per second
sum by (vmid, device) (rate(blockstat_rd_bytes[5m]))
sum by (vmid, device) (rate(blockstat_wr_bytes[5m]))
```

Reduce each to one scalar over the decision window with a robust quantile rather than a mean, so a single spike neither triggers nor suppresses a migration:

```
quantile_over_time(0.95, ( <expression above> )[24h:5m])
```

Notes for the implementer:

- `rate()` already handles the counter resets that occur when a VM reboots or migrates between nodes.
- The `sum by` collapses the `nodename`/`host` labels, which is what we want: a VM that live-migrated between nodes during the window must remain **one** disk, not two.
- Use the **range** (`query_range`) form as well when the forecaster needs a series rather than a scalar; the same expression works with a `step`.
- Reject a disk whose sample coverage over `W` is below `window.min_coverage` and fall back to its last known load from `state.json`, flagging it in the plan output. Never treat missing data as zero load — that would silently invite migrations *onto* a busy storage.
- **Scope every query to this cluster's own nodes.** None of the expressions above restrict which series they match beyond the metric name itself, which is fine when one Prometheus serves exactly one PVE cluster but silently wrong the moment it serves more than one (or anything else emitting a same-named metric): `vmid` is only unique *within* a cluster, so an unscoped query would sum a same-numbered vmid from somewhere else into this one's load without any error or warning. Add a matcher inside the vector selector, before `rate()`: `<metric>{nodename=~"pve01|pve02|..."}`, built from the cluster's own node list (`GET /nodes`, §3.5) every run rather than hand-maintained, so a node added to the cluster is covered with no config edit. `metrics.extra_selector` lets the operator override this outright with a raw PromQL matcher of their own — needed when Telegraf's tagging doesn't carry PVE's own node name verbatim, or the restriction needed is something else entirely (a `cluster` tag, say, distinguishing which of several PVE clusters a series belongs to). `pve-storage-drs verify-metrics` applies only the override, never the auto-derived filter: it is deliberately independent of the PVE API, so it has no node list to build one from.

3.5 PVE API

`pve.py` is built on `proxmoxer`, not a hand-rolled ticket/CSRF client. `proxmoxer` implements the ticket exchange and API-token auth below itself, behind a `ProxmoxAPI` object whose attribute chaining (`proxmox.nodes(node).qemu(vmid).config.get()`) maps directly onto the endpoint table below. The decisive reason is its **backend abstraction**: the same `ProxmoxAPI` interface is available over plain HTTPS (`backend="https"`, the default and the only one this project uses today), or over SSH — either `openssh` (shells out to the system's own `ssh` + `pvesh`) or `ssh_paramiko` (an in-process SSH client). A deployment that cannot or will not open tcp/8006 to the management host can switch to an SSH-based backend as a **configuration change in one factory function** (`pve.build_client`), with no change to `PveClient`'s methods or to anything that calls them — exactly the shape a hand-rolled HTTPS-only client would not have offered. `python3-proxmoxer` is packaged for Debian trixie (§2.1).

Authentication, as `proxmoxer`'s `https` backend performs it (username/password as originally specified, API token preferred and what this project actually configures by default):

```
POST /api2/json/access/ticket        {username, password}
  → data.ticket                      → Cookie: PVEAuthCookie=<ticket>
  → data.CSRFPreventionToken         → header CSRFPreventionToken on every write
```

Tickets are valid ~2h; `proxmoxer` refreshes them itself on its own internal interval when using password auth. API tokens (`Authorization: PVEAPIToken=USER@REALM!TOKENID=SECRET`) need no CSRF header, no refresh, and are preferred for unattended `auto` mode; `proxmox.auth.token_id`'s `user@realm!tokenname` format is split into `proxmoxer`'s separate `user`/`token_name` arguments at the one place `pve.py` constructs the client.

Read path:

Endpoint	Used for
`GET /cluster/resources?type=vm`	VM inventory: vmid, node, status, name, tags
`GET /cluster/resources?type=storage`	Storage inventory, `shared` flag, used/total per node
`GET /storage`	Storage definitions: type, `content`, `shared`, `nodes` restriction, **and** per-storage `saferemove` / `saferemove_throughput` — see §7.1 and §9.3
`GET /nodes`	Every node in the cluster, by name — §3.4's PromQL node-scoping filter, independent of which nodes currently host a VM or shared storage
`GET /nodes/{node}/qemu/{vmid}/config`	**disk → storage mapping and size**
`GET /nodes/{node}/storage/{storage}/status`	authoritative `total`/`used`/`avail`
`GET /nodes/{node}/storage/{storage}/content`	per-volume real allocated sizes, owner vmid
`GET /nodes/{node}/qemu/{vmid}/snapshot`	detect existing snapshot/volume chains (§3.7)
`GET /nodes/{node}/qemu/{vmid}/status/current`	`lock` state immediately before a move (§9.3)

Use `GET /storage` (the list form), never `GET /storage/{storage}`, for a storage's own config including `saferemove`. Verified empirically against a live PVE 9.2.11 cluster: an API token granted only `Datastore.Audit` can list every storage's full config via `GET /storage`, but the same token gets `403 Forbidden (Datastore.Allocate)` calling `GET /storage/{storage}` for the exact same storage — the single-item form apparently backs an edit-UI code path gated by the ability to change the config, not merely read it. The list form returns the identical per-storage object for every storage in one call, so nothing is lost by preferring it; it is also one call instead of `|storages|` calls.

`GET /nodes/{node}/storage/{storage}/content` itself needs `Datastore.Allocate`, not merely `Datastore.Audit`, contrary to what an "Audit-only, read-planning tool" design would hope. Also verified empirically on the same live cluster: with only `Datastore.Audit` granted, the call succeeds (HTTP 200) but returns an empty list on every storage, node and content-type filter tried, with no error at any level — a false negative that looks exactly like "this storage has no content" unless you already know to be suspicious of it. Granting `Datastore.Allocate` (bundled with `Datastore.Audit` in a custom role, scoped per-storage — see below) on the same storages made `/content` immediately start returning real data, retested and confirmed. So the tool's true minimum privilege per storage in scope is `Datastore.Audit` **and** `Datastore.Allocate`, not Audit alone; `move_disk` needs nothing beyond that on the storage side (the VM side needs `VM.Config.Disk` and `VM.Migrate` or equivalent, out of scope for this note). Document this precisely in the operator manual rather than repeating the more comfortable but wrong "Audit is enough" claim — an operator who grants only Audit will see the tool silently treat every storage as empty of tracked volumes, which corrupts the U^{sext} accounting of §5.1.1 without ever raising an error.

Two more things this exposed about Proxmox's ACL model, worth knowing before configuring a token: first, **API tokens have their own ACL entries, separate from their owning user** — with privilege separation enabled (the default), a token's effective permission is the *intersection* of the user's permissions and whatever is granted to the token specifically, so granting a role to the user alone has no effect on the token, and granting it to the token alone has no effect either unless the user has it too. Both need the grant. Second, ACL entries at `/storage` (the parent path, with `propagate: 1`) were not sufficient in practice to make `Datastore.Allocate` retroactively work on already-listed sub-paths in one observed sequence during testing — granting the role explicitly at each `/storage/{id}` resolved it; whether the parent-path grant would have converged given more time was not conclusively isolated, so the operator-facing guidance below grants per-storage explicitly rather than relying on propagation from `/storage`.

Parsing a disk from the VM config: a key matching

```
^(?:ide[0-3]|sata[0-5]|scsi(?:[0-9]|[12][0-9]|30)|virtio(?:[0-9]|1[0-5])|efidisk0|tpmstate0|unused\d+)$
```

whose value looks like `san-a:vm-101-disk-0,size=512G,iothread=1`. The storage id is the part before

the first `:`; the size comes from the `size=` parameter, cross-checked against `/storage/{storage}/content`, which is authoritative for what is actually allocated. **Every bus counts** — a VM's boot disk on `ide0` or `sata0` occupies and loads a storage exactly as a `scsi0` does, and a tool that enumerates only `scsi*` will silently mis-account capacity and produce plans that cannot reach balance. §3.6 covers which of these can actually be moved.

Skip only entries whose value contains `media=cdrom`. That covers ISO mounts, empty drives (`none,media=cdrom`) and cloud-init drives (`san-a:vm-101-cloudinit,media=cdrom`). A cloud-init volume does occupy real bytes on the storage, so it is not in `D` but it **is** counted in U^{ext} (§5.1.1) like any other volume DRS does not manage.

Disk format. Detect the source format from the volume returned by `/storage/{storage}/content` (`format: raw|qcow2|…`), falling back to the storage type's default (raw for LVM, qcow2 for directory storages, raw for ZFS zvols). (C2) fixes `x_{d,s} = 0` when `s` cannot store that format. `move_disk` without an explicit `format` preserves the source format, which is what we want: **format conversion is out of scope and must stay disabled by default.** Only pass `format=` when an operator has explicitly opted in — converting raw→qcow2 on shared LVM is what enables volume-chain snapshots, but it is a deliberate storage-policy change, not something a balancer should do silently.

`pve-storage-drs verify-storages`. A companion to `verify-metrics` (§3.3), run once per storage before relying on any plan. For every storage in every group it reports `type`, `shared`, `content`, `saferemove`, `saferemove_throughput`, total/used, and the largest disk currently on it; then it derives the implied wipe time `z_max / saferemove_throughput` and warns when that exceeds `migration.max_single_move_duration` or `gates.cooldown_per_storage` (§9.3). It also prints the expansion of every `/…/` storage pattern (§11.4) — the entry and the storages it matched — and lists cluster storages matched by no group, so an over-broad or dead pattern is visible before any plan relies on it. This is the command that turns "why has this balancer been stuck for two days" into a line of output before the first move.

Read-path cost and concurrency. The topology read is `O(number of VMs)`: PVE has no batch config endpoint, so `/qemu/{vmid}/config` must be fetched per VM. For a few hundred VMs, serial fetching dominates run time. Specify:

- a bounded thread pool (8–16 workers, configurable) for the per-VM config fetches, with a short per-request timeout and bounded retries on 5xx;
- a **per-run topology cache** — one snapshot at the start of the run, reused by the load model, solver and scheduler;
- the pre-move re-validation in §9.2 **must bypass this cache** for the specific VM and target storage it is about to touch, and only for those; everything else may be reused;
- `/cluster/resources` is a single call and gives the VM inventory, node placement and coarse storage usage, so fetch it first and use it to decide which per-VM configs are needed at all — in practice this means only a **stopped** VM, when `exclude.running_only` is set, can be skipped without fetching its config, since that is the only exclusion `cluster/resources`'s own fields (`status`) can decide. A VM excluded by `exclude.vmids`/tags still needs its config fetched: (C2) *pins* such a disk into `D` rather than dropping it (§5.1.1's note on why), which needs its size. A disk turning out to be "ungrouped" (§3.6) is only discoverable *after* fetching the config that reveals which storage it is actually on, so it costs a fetch too — the saving from this optimization is real but smaller than a naive reading suggests.

Expected call count per run: `3 + 2·|VMs considered| + 2·|storages|` — three cluster-wide calls (VM inventory, storage inventory, storage definitions), two per considered VM (config, which also carries `lock` per §9.3's pseudocode so no separate `/status/current` call is needed at planning time; and `/snapshot`, per §3.7), and two per storage (`status`, `content`).

Write path:

```
POST /api2/json/nodes/{node}/qemu/{vmid}/move_disk
     disk=scsi0  storage=san-c  delete=1  bwlimit=<KiB/s>  [format=qcow2]
  → UPID string
GET  /api2/json/nodes/{node}/tasks/{upid}/status   → status=running|stopped, exitstatus=OK|…
```

`bwlimit` is in **KiB/s**. `delete=1` removes the source volume after a successful mirror; without it the old volume is left behind as an unreferenced volume and the reserve math will silently drift.

3.6 Which disks can actually be moved online

Enumerating every bus (§3.5) is necessary but not sufficient: the set of disks that *exist* is larger than the set that can be relocated while the VM is running, and the difference is load-bearing for both the capacity model and the affinity objective.

Config key	In `D` (movable)?	Why
`ide0-3`, `sata0-5`, `scsi0-30`, `virtio0-15`	**yes**	ordinary QEMU block devices; `move_disk` mirrors them online
`efidisk0`	**yes**	movable online — verified on PVE 9.2. Tiny (a few MiB), so its migration cost is effectively zero
`tpmstate0`	**yes**	movable online — verified on PVE 9.2. Tiny. PVE may not use the `drive-mirror` path for it, since `swtpm` rather than QEMU owns the state; see the note below
`unused0-N`	**yes**, with $\ell\_d = 0$	a real allocated volume detached from the VM. It occupies bytes and counts against the reserve, but generates no I/O
anything with `media=cdrom`	no — not in `D`, counted in U^{sext}	ISO mounts, empty drives, cloud-init volumes

`efidisk0` and `tpmstate0` move online. This was confirmed empirically on a live PVE 9.2 cluster by the operator. Older Proxmox forum threads (PVE 6.x/7.x era) report online moves of these two failing, and that history is worth knowing only so nobody re-derives an obsolete restriction from a search result: it does not apply to 9.2. They are ordinary members of `D`.

Two practical notes. First, they are **small** — an EFI var store is a few MiB, TPM state likewise — so $\gamma \cdot z\_d$ and the §7 payback cost are negligible for them, while β charges a full migration for each. A plan that drags a 4 MiB `efidisk0` across the group to satisfy κ therefore pays a real move-count penalty for near-zero bytes; that is the correct accounting (it *is* a task, with task overhead and a lock window), but it is the reason to tune β and κ together rather than in isolation. Second, PVE may not use the `drive-mirror` path for `tpmstate0`, because `swtpm` rather than QEMU holds that state. Do not depend on drive-mirror semantics for it. The transient invariant of §8.1 — the volume occupies **both** storages until the move completes — is the conservative assumption and stays correct under either mechanism, so no part of the model needs to know which one PVE picked. Be clear about which way that conservatism cuts: if `swtpm` does *not* use drive-mirror, the both-storages assumption over-reserves during the move. Over-reserving can never cause a reserve breach, so it is safe; the only cost is that a `tpmstate0` move onto a nearly-full storage may fail the transient check when it would physically have fitted. Treat that exactly like any other infeasible move — **defer it, never force it** — and let `pve-storage-drs explain` say that the transient check was the blocker. Given that TPM state is a few megabytes, a storage tight enough for this to bind is a storage with a much larger problem.

Metrics coverage differs by device type. `efidisk0` is a QEMU drive and should appear in `blockstat`; `tpmstate0` and `unused{N}` are not QEMU block devices, so no series will exist for them and $\ell\_d = 0$ under

the `min_coverage` rule of §3.4. That is correct rather than a gap — they generate no guest I/O worth balancing. Have `pve-storage-drs verify-metrics` report which config keys resolved to a series and which did not, and classify these as **expected-absent** rather than as errors, so a genuine coverage problem on a `scsi0` still stands out. Treat the exact per-device-type coverage as a thing to observe on your cluster, not to assume from this table.

Pinned disks are modelled, not ignored. Disks pinned by (C2) — snapshot-blocked (§3.7), config-excluded, or locked this run — enter the MILP as ordinary disks with $x_{d,\sigma_0(d)} = 1$ fixed. This is deliberately *not* the same as excluding them: their bytes must still count toward $\Sigma_d\, z_d \cdot x_{d,s}$ and toward Z_s in the reserve constraint (C5), and their load toward L_s. Treating them as foreign volumes instead would work for capacity but would lose the fact that they belong to a VM whose other disks we are placing.

Pinned disks are excluded from the affinity term by default. κ (§5.4) counts a VM's spread over storages. If an immovable disk counted, a VM with one snapshot-blocked volume would be permanently "fragmented" the moment any other disk moved, and κ would veto good placements to chase a co-location that cannot be achieved this run. So the $y_{v,s}$ linking of (C3) ranges over **movable** disks only unless `objective.affinity_counts_pinned_disks` is set. Both behaviours are defensible; the default is the one that does not let an unreachable disk dictate placement of the rest.

Unused disks move only to repair the reserve, and that is correct. They carry $\ell_d = 0$ — no series exists for a volume QEMU has not opened — so relocating one yields zero imbalance benefit while incurring the full $\gamma \cdot z_d$ byte penalty and the full payback cost of §7. The solver will therefore leave them alone until a capacity constraint forces the issue, which is exactly the desired policy. Set `exclude.include_unused_disks: false` to pin them instead; they then count via U^{ext}.

An `unusedN` volume can be absent from `GET /storage/{s}/content` — not PVE's normal behaviour, but not rare enough to ignore either. Found on a live PVE 9.2.11 cluster on Ceph RBD storage: a VM's `unused0` entry named a volume (`VM:vm-104-disk-2`) that PVE's own config still tracked, but that volume did not appear anywhere in that storage's content listing, while the same VM's two active (`scsiN`) disks did. The operator confirmed the cause: the underlying volume had been removed directly on the storage backend, outside Proxmox, leaving `unused0` a dangling reference in the VM's config. This is not something PVE catches on its own — an `unusedN` entry is not validated at VM start the way an attached disk is, so the VM boots normally with the stale reference still sitting in its config. `topology.py` treats a content-listing gap the same regardless of cause (§3.5): it falls back to the VM config's own `size=` for that disk and logs a warning naming it as unauthoritative, which is the conservative direction to be wrong in here — believing a since-deleted volume still occupies its former space costs nothing but a temporarily pessimistic reserve calculation, while the reverse (silently dropping it) could let a real volume's bytes go uncounted. An operator seeing this warning should treat it as a prompt to check for, and if confirmed gone, clean up the dangling reference (e.g. `qm unlink <vmid> unusedN`) rather than something the tool should silently paper over.

Evacuating a storage completely is therefore possible online — every disk type in the table above except CD-ROM-media entries can be relocated with the guest running, and CD-ROM entries hold no storage-owned data except cloud-init volumes, which are regenerable. Where a full evacuation is *not* achievable it is because of a §3.7 snapshot or an explicit exclusion, never because of a device type. `pve-storage-drs explain` must name the actual blocker per VM — *"106: cannot fully consolidate, 2 snapshots on scsi0"* — rather than emitting a plan that quietly leaves a stray volume behind.

3.7 Disks with snapshots are excluded, loudly

`move_disk` is issued with `delete=1` throughout (§3.5), and PVE refuses that combination on a volume that has snapshots — *"you can't move a disk with snapshots and delete the source"*. Even where a move is

accepted, PVE does not carry the snapshots across: they are left behind or lost. Under PVE 9 volume-chain snapshots the situation is worse, because a snapshot is a *separate full-size volume* on the source storage, so a "moved" disk would leave most of its bytes behind and the reserve arithmetic would silently drift.

There is no safe automatic remedy. The two real options both belong to the operator: delete the snapshots (a data-retention decision the balancer must not make), or move the disk offline with the VM shut down (out of scope — this tool never stops a VM, §1). So the policy is **skip, and complain**:

1. **Detect per volume, not per VM.** `GET /nodes/{node}/qemu/{vmid}/snapshot` is authoritative for VM-level snapshots and pins *every* disk of that VM. Independently, cross-check `/storage/{s}/content` for volumes owned by the VM that its current config does not reference: under volume-chain storages those are chain members, and elsewhere they are orphans. Either way the disk is unsafe to move. Do not rely on volume-name patterns — they are storage-specific.
2. **Pin, do not drop.** An excluded disk stays in the model with $x_{d,\sigma_0(d)} = 1$ so its bytes and load remain accounted for in (C4)/(C5)/(C6); it is not demoted to a foreign volume.
3. **Complain, every run, at WARN.** List each affected VM with its pinned bytes and pinned load, and the group total of both. A one-line "3 VMs skipped" is not enough: the operator needs to know *which* snapshots to clear to unblock balancing.
4. **Say when the goal has become unreachable.** If pinned load exceeds `report.warn_pinned_load_fraction` of a group's total (default 0.25), the residual imbalance may be structural rather than a planning failure. Report the best achievable spread *given the pins* alongside the actual one, so a stubborn 40% spread is attributable to snapshots rather than looking like a broken solver.

`exclude.skip_vms_with_snapshots` stays as a knob but its `false` setting does not make such moves work — it merely stops pre-filtering them, and PVE will reject them at the API. Keep it `true`; the pre-flight check of §9.3 runs regardless.

---

4. The load model

For each disk d, reduce the three raw quantities to a single scalar ℓ_d, expressed in **average in-flight I/O requests**. The three terms have wildly different magnitudes, so they are normalized against each other before weighting and the blend is then rescaled back onto the in-flight-I/O scale; the units matter downstream and are spelled out below.

Read and write are combined **before** normalization, using the configurable asymmetry factors ρ = `load_weights.read_factor` and ω = `load_weights.write_factor` (both 1.0 by default). Reads and writes are therefore fetched as **separate series** and combined engine-side in `loadmodel.py`, not summed inside PromQL — this keeps the factors in tested code rather than in deployed query strings, and makes them unit-testable in isolation:

$$
\begin{aligned}
\mathrm{raw\_t}(d) &= \rho \cdot \mathrm{rd\_time\_d} + \omega \cdot \mathrm{wr\_time\_d} && \text{(in-flight I/O, s/s)} \\
\mathrm{raw\_o}(d) &= \rho \cdot \mathrm{rd\_ops\_d} + \omega \cdot \mathrm{wr\_ops\_d} && \text{(ops/s)} \\
\mathrm{raw\_b}(d) &= \rho \cdot \mathrm{rd\_bytes\_d} + \omega \cdot \mathrm{wr\_bytes\_d} && \text{(bytes/s)}
\end{aligned}
$$

Each term is then normalized by its group total, because the three have wildly different magnitudes (in-flight I/O ≈ 0–10, ops/s ≈ 0–50 000, bytes/s ≈ 0–10^9) and unnormalized weights would be meaningless. Normalization makes the three terms **commensurable**; it must not be the last step, because on its own it also destroys the physical meaning of the result. So blend in normalized space, then rescale back onto the in-flight-I/O scale:

$$
i_d = \frac{\mathrm{raw\_t}(d)}{\sum_{e \in g} \mathrm{raw\_t}(e)}, \quad o_d = \frac{\mathrm{raw\_o}(d)}{\sum_{e \in g} \mathrm{raw\_o}(e)}, \quad b_d = \frac{\mathrm{raw\_b}(d)}{\sum_{e \in g} \mathrm{raw\_b}(e)}
$$

$$\ell_d = T_g \cdot \frac{w_t \cdot i_d + w_o \cdot o_d + w_b \cdot b_d}{w_t + w_o + w_b} \qquad \text{with } T_g = \Sigma_{e \in g}\ \text{raw}_t(e)$$

with `w_t = 1, w_o = w_b = 0` by default — **I/O time is the primary quantity**.

Units of ℓ, and why the rescale is not cosmetic. `T_g` is the group's total average in-flight I/O, so $\Sigma_{d \in g}\ \ell_d = T_g$ exactly and ℓ_d is measured in **average in-flight I/O requests** — the same physical unit as `raw_t`. Under the default weights the rescale is an exact identity, $\ell_d = \text{raw}_t(d)$. Everything downstream depends on this absolute scale:

- §7 compares a plan's benefit against a mirror's cost, where $\omega = 1.0$ means *one* sequential reader or writer in flight. That comparison is only valid if ℓ is on the same absolute scale. Were ℓ left normalized to sum to 1, $\omega = 1.0$ would silently be `T_g` times too large — on a busy group with $T_g \approx 20$ every migration would look twenty times more expensive than it is and the payback test of §7.3 would reject almost everything.
- §7.3's saturation guard and §5.3's big-M bound both need an absolute load scale.
- The worked example in §14 uses raw loads ($\Sigma\ell = 7.4$, not 1.0) and is self-consistent only under this definition.

Only *ratios* between disks matter to the balance objective, so the rescale changes no optimal assignment; it changes every quantity that is compared against a physical constant. Guard the divisions: if a group's total for a term is 0 that term contributes 0 for every disk rather than producing NaN, and if `T_g = 0` the entire group is idle — skip it, there is nothing to balance.

The objective weights of §5.4 are therefore calibrated in units of in-flight I/O per storage, which is what makes $\alpha = 1.0$ against $\beta = 0.25$ a meaningful default pair: a migration must buy at least a 0.25-request reduction in summed deviation to be worth making.

Why I/O time is the right default. `rate(rd_total_time_ns + wr_total_time_ns) / 1e9` is, by Little's law, the **average number of I/O requests in flight** for that disk. It is dimensionless, it is additive across disks on the same storage, and it is directly comparable between storages of different size and speed. Crucially it *self-weights*: a 1 MiB write that takes 8 ms contributes eight times what a 4 KiB read taking 1 ms does, without anyone having to guess a read/write weight. Pure IOPS treats those two operations as equal and so systematically under-counts large sequential load; pure throughput does the reverse and under-counts small random load. `ops` and `bytes` remain available as additional weighted terms for operators who want to express a policy the array's own timings do not capture.

Because I/O time already embodies the real cost asymmetry between reads and writes, leaving `read_factor = write_factor = 1.0` is correct for the default configuration. The factors exist for the `ops`/`bytes` terms, where the asymmetry is *not* otherwise represented — a write to a RAID-6 array costs far more than a read of the same size, and neither an operation count nor a byte count knows that. Applying them to the I/O-time term as well is supported but double-counts, so do it only deliberately.

One caveat to document: I/O time is a *feedback* signal — it rises when the array is slow, including when the slowness is caused by some other tenant. Combined with the drift gate and cooldowns this is stable in practice, but an operator seeing oscillation should shift weight toward `ops`/`bytes`, which measure only what the guest requested.

Storage load and utilization, where `c_s` is the configured capability weight:

$$L_s = \Sigma_d\ \ell_d \cdot x_{d,s} \qquad u_s = L_s / c_s$$

`u_s` — not `L_s` — is what gets equalized, so a storage with half the spindles can be given `capability_weight: 0.5` and will correctly be assigned half the load.

---

5. The optimization problem

Solved **independently per group** — groups share no variables and no constraints, so a cluster with four groups is four small problems, not one large one.

5.1 Sets and data

Symbol	Meaning
`g`	a storage group
`S`	storages in the group
`D`	movable disks currently in the group
`V`	VMs owning at least one disk in `D`
`v(d)`	the VM owning disk `d`
$\sigma_0(d)$	the storage disk `d` is on **now**
`z_d`	provisioned size of disk `d` (bytes)
ℓ`_d`	load of disk `d` (section 4)
`C_s`	total capacity of storage `s`
U^{ext}	bytes on `s` consumed by volumes DRS does not manage (§5.1.1)
`c_s`	capability weight of `s`
`f_s`	snapshot reserve factor for `s` (default 2.0)
`N_s`	saturation load of `s`, in in-flight I/O requests; optional, §7.3 only — not part of the MILP

5.1.1 Computing U^{ext}

```
U^ext = Σ { size(vol) : vol ∈ GET /nodes/{node}/storage/{s}/content ,
                        identity(vol) ∉ D }
```

where `identity(vol)` is the `(vmid, device)` pair the volume belongs to, resolved by matching the volume id against the owning VM's config, and `D` here means *every* disk (C2)'s eligibility pass tracks for this group, pinned or not — `Σ_{s∈S} x_{d,s} = 1` holds for `d ∈ D` regardless of whether a specific `x` is fixed. Everything that is **not** a member of `D` counts as foreign: templates, ISOs and backups, disks of stopped or ungrouped VMs (§3.6/(C2): never fetched, never entered into any `D` at all), disks belonging to another group that shares the storage, unreferenced orphans, and **orphaned target volumes left by a previously failed move** (§9.3). Counting those orphans is intentional — they really do occupy the LUN, and letting them inflate U^{ext} is what makes their cost visible rather than silently eroding the reserve.

Config-excluded disks (`exclude.vmids`/`exclude.disks`/`tags`/`no-drs`) are *not* on this list. Section 5.3 (C2) pins them into `D` rather than dropping them, precisely so their bytes still count toward `Σ_d z_d·x_{d,s}` in (C4)/(C5) and their fragmentation toward κ — see "Pinned disks are modelled, not ignored" in §3.6. Treating a config-excluded disk as foreign instead would double the inconsistency: it would still occupy the reserve calculation correctly by accident (foreign bytes are also subtracted from capacity) but would silently break the affinity accounting for the rest of that VM's disks, which is exactly the failure mode §3.6's rule exists to prevent. An earlier draft of this section listed "excluded" alongside "stopped" and "ungrouped" here, which was a direct contradiction of (C2) rather than a second valid path — fixed in the same commit that first implemented this join in `topology.py`.

Set $U^{ext} = 0$ only if `snapshot_reserve.count_foreign_volumes` is false, which is not recommended.

5.2 Variables

Variable	Domain	Meaning
x_{d,s}	{0,1}	disk d is placed on storage s
y_{v,s}	{0,1}	VM v has at least one disk on s
Z_s	≥ 0	size of the largest disk on s
e_s	≥ 0	absolute deviation of u_s from target (L1 objective)
t	≥ 0	maximum utilization (min–max objective)
r_s	≥ 0	reserve violation slack

5.3 Constraints

(C1) Assignment. Every disk lands on exactly one storage in its group:

```
Σ_{s∈S} x_{d,s} = 1                                   ∀ d ∈ D
```

(C2) Eligibility. Fix `x_{d,s} = 0` wherever placement is impossible or forbidden. This is where all the domain rules live, and doing it as variable fixing rather than as constraints keeps the model small:

- `s` does not have `images` in its `content` list;
- `s` is not shared, or is restricted to nodes that cannot see the VM;
- `s` cannot hold the disk's format;
- `d` or `v(d)` is excluded by config (`exclude.vmids`, `exclude.disks`, tags, `no-drs`) — also pin;
- $\sigma_0(d)$ belongs to **no** configured group — such a disk is unmanaged: it is not in any `D`, it is pinned where it is, and its bytes count toward U^{ext} of its storage. Report it in `show-load` as "ungrouped, not managed" so an unintended omission from `groups` is visible rather than silent;
- `d`, or any disk of `v(d)`, has a snapshot or an unreferenced companion volume on its storage (§3.7) — pin $x_{d,\sigma_0(d)} = 1$. `move_disk delete=1` cannot move such a volume and would not carry the snapshots if it could;
- `d` is within its per-disk cooldown — also pin to current;
- `v(d)` is currently `locked` (§9.3) — pin for this run. A lock is transient, so this is a *planning-time* pin only and carries no cooldown; the next run re-evaluates it.

(C3) VM affinity linking. Couple `y` to `x` in both directions so the objective term is exact. Let `D^mov ⊆ D` be the disks that are not pinned by (C2):

```
x_{d,s}  ≤  y_{v(d),s}                                ∀ d ∈ D^mov, s ∈ S
y_{v,s}  ≤  Σ_{d ∈ D^mov : v(d)=v} x_{d,s}            ∀ v ∈ V, s ∈ S
```

Ranging over `D^mov` rather than `D` keeps a disk that cannot move this run — snapshot-blocked, config-excluded or locked — from dictating where a VM's movable disks may go (§3.6). Set `objective.affinity_counts_pinned_disks: true` to range over all of `D` instead, which is the right choice only if you would rather chase an unreachable co-location than balance well.

(C4) Largest-disk linearization. `Z_s = max{ z_d : x_{d,s}=1 }` is not linear, but because the reserve constraint pushes `Z_s` *down* while this pushes it *up*, a one-sided bound is exact at the optimum:

```
Z_s  ≥  z_d · x_{d,s}                                 ∀ d ∈ D, s ∈ S
```

(C5) Capacity and snapshot reserve. The core safety constraint. The reserve is the **larger** of the snapshot term and the configured flat floor, so introduce `R_s ≥ 0`:

```
R_s  ≥  f_s · Z_s
R_s  ≥  min_free_bytes_s                                        (constant)
```

```
Σ_d z_d·x_{d,s}  +  U^ext  +  R_s   ≤   C_s  +  r_s              ∀ s ∈ S
```

Two one-sided bounds are exact for `R_s = max(f_s·Z_s, min_free_bytes_s)` because (C5) pushes `R_s` *down* while both bounds push it *up*. The `f_s · Z_s` term is the "always keep 2× the largest disk free" rule, and (C4) is what makes it expressible in a linear model at all. `min_free_bytes` is the absolute floor for a storage whose largest disk is small — with `f=2` and a 10 GiB largest disk, the snapshot term alone would reserve only 20 GiB on a 20 TiB LUN.

`r_s` is a **repair slack**, not a licence to overfill. A storage can already be violating the reserve when the engine first runs (see the worked example in §14), and a hard `≤ C_s` would make the model infeasible and the tool useless exactly when it is most needed. Two ways to keep it effectively hard:

1. **Lexicographic, preferred and provably correct.** Solve in two stages: minimize `Σ_s r_s` alone; then fix `Σ_s r_s` to that minimum as a constraint and minimize the §5.4 objective. Both CP-SAT and CBC support this by re-solving. The reserve is then never traded against balance at any weight, and `Σ r_s > 0` provably means *physically impossible*, not merely *unattractive*.
2. **Single-stage big-M**, simpler but requiring calibration: keep `P · Σ_s r_s` in the objective with `P` large enough that no achievable gain from the other terms can pay for a violation worth caring about. `P` must be **computed at model-build time, not taken from config as a fixed number**, because the bound depends on the group's absolute load `T_g` (§4):

   ```
   U_obj  =  2·α·T_g  +  β·|D|  +  γ·Σ_d z_d  +  κ·|V|·(|S|−1)      (upper bound on the
                                                                      non-reserve objective)
   P_min  =  U_obj / ε_r          with  ε_r = the smallest reserve shortfall we refuse to trade
   ```

   `Σ_s e_s ≤ 2·T_g` because every `u_s` lies in `[0, T_g/c_s]` and the deviations from `u*` sum to at most twice the total. Take `ε_r = 1 MiB` expressed in the same size unit as `z_d` (2^{-20} TiB if sizes are TiB), which says: never accept even a one-mebibyte reserve shortfall in exchange for balance. Config `objective.reserve_violation_penalty` is then a *floor*, not the value used: the model uses `P = max(configured, P_min)` and logs a warning when it had to raise it. That warning must be *checkable*, not just an announcement: log `P_configured`, `P_min`, `P_used`, and the four inputs the bound came from — `T_g`, `|D|`, `Σ_d z_d`, `|V|·(|S|−1)` — plus the `ε_r` granularity, and repeat them in `pve-storage-drs explain`. `P_min` moves with `T_g`, so the same config file legitimately yields different effective penalties on a quiet group and a busy one, and on the same group at different times of day. An operator who sets `reserve_violation_penalty: 5000` and sees the engine using 2.3×10^7 needs to be able to reconstruct that number rather than take it on faith.

   Worked against §14 (`T_g = 7.4`, `|D| = 6`, `Σz = 6.5 TiB`, `|V| = 5`, `|S| = 3`, sizes in TiB): `U_obj = 14.8 + 1.5 + 0.325 + 5.0 = 21.6`, so `P_min = 21.6 · 2²⁰ ≈ 2.27×10⁷`. The configured default `P = 1000` is **four orders of magnitude too small** to be provably dominant at mebibyte granularity — it is dominant for violations above roughly 22 GiB and silently tradeable below that. This is precisely why option 1 is the default and this option needs the computed `P`.

Use the lexicographic solve (option 1) by default. It needs no calibration, its correctness does not depend on `T_g`, and both backends support it by re-solving. Option 2 exists for a backend that cannot re-solve cheaply.

As built (REVIEW.md V-01): option 2's `P_min` computation and `max(configured, P_min)` floor above were never implemented — there is no build-time computation of a provably-dominant `P` anywhere in `src/`. Both MILP backends (`optimize.py`) implement option 1 only, and never consult `objective.reserve_violation_penalty` at all. The heuristic backend (`heuristic.py`) is the key's one live consumer: it multiplies the *configured* value straight into its objective as `reserve_penalty_term = objective.reserve_`

`violation_penalty * reserve_shortfall_tib`, with no floor and no warning if it is too small to be provably dominant for a given group. Until option 2 is implemented, read this section's `P = max(configured, P_min)` machinery as the target design for the single-stage path, not current behaviour — see `docs/manual/10-configuration.md`'s `objective.reserve_violation_penalty` entry for what the key does today.

`tests/fixtures/reserve-tradeoff.yaml` (§14.6) is the fixture for this rule: a two-storage group in which the two options provably disagree, with the exact `P` at which big-M flips recorded alongside the computed `P_min`. Test both paths against it. The §14 fixture cannot do this job — there, every reserve-violating assignment is also worse on balance, so both options agree at any `P`.

Report any residual `r_s > 0` prominently as an unfixable shortfall, with the byte amount.

(C6) Spread. With `u* = (Σ_d ℓ_d) / (Σ_s c_s)` — a **constant**, since total group load is invariant under reassignment — the L1 form is fully linear:

```
u_s = (Σ_d ℓ_d·x_{d,s}) / c_s
u_s − u*  ≤  e_s        and        u* − u_s  ≤  e_s     ∀ s ∈ S
```

The min–max alternative is `t ≥ u_s ∀s`. L1 is the default: min–max only sees the single hottest storage and is indifferent to everything below it, which tends to produce plans that fix the worst storage and ignore a second nearly-as-bad one.

5.4 Objective

```
min   α · Σ_{s∈S} e_s                              (imbalance)
    + β · Σ_{d∈D} (1 − x_{d,σ₀(d)})                (number of migrations)
    + γ · Σ_{d∈D} z_d · (1 − x_{d,σ₀(d)})          (bytes migrated)
    + κ · Σ_{v∈V} ( Σ_{s∈S} y_{v,s} − 1 )          (VM disk fragmentation)
    + P · Σ_{s∈S} r_s                              (reserve violation)
```

`(1 − x_{d,σ₀(d)})` is exactly 1 when disk `d` moves and 0 when it stays, so β directly implements "minimize the number of migrations" and γ biases against moving *large* disks specifically. κ counts the number of **extra** storages a VM is spread across, so it is 0 for a VM whose disks are all together and grows by 1 per additional storage — a soft preference that free space (C5) or a strong imbalance can legitimately override, as required.

κ measures **within-group** fragmentation only. Because the problem decomposes per group and a disk can never leave its group, a VM with disks in two different groups is not counted as fragmented — that spread is structural and no migration could ever repair it. This is a consequence of the decomposition, not an oversight.

Scaling matters: express `z_d` in TiB and `ℓ_d` in average in-flight I/O requests (§4) before applying the weights, so the defaults in the example config are meaningful.

5.5 Solver backends

CP-SAT (preferred). All variables and coefficients must be integral. The single most important observation is that `ℓ_d`, `z_d`, `c_s`, `C_s` and U^{ext} are **data, not variables** — every one of them appears only as a coefficient multiplying a binary. So they never need a shared scale factor of their own: fold each of them into its coefficient *once*, at full precision, and round the finished coefficient. Doing that removes both classes of scaling error that a naive "scale everything by K" approach introduces.

Two scales are needed, one for quantities that appear as *variables* and one for the objective:

Scale	Applies to	Value
K	the load-valued variables e_s, t, and the constants u*, L_s they are compared against	10^6 (micro-requests)
—	the size-valued variables Z_s, R_s, r_s and the constants z_d, C_s, U^{ext}, min_free_bytes	MiB (integers already)
W	every objective weight, so α, β, γ, κ, P keep four decimals	10^4

Constraint coefficients. In (C6) the storage load enters as Σ_d ℓ_d·x_{d,s} / c_s. Do *not* compute round(K/c_s) and multiply — that rounds the capability weight itself and makes CP-SAT and CBC disagree for non-binary c_s. Fold both constants into one per-(disk, storage) coefficient:

```
a_{d,s} = round(K · ℓ_d / c_s)          →   Σ_d a_{d,s}·x_{d,s} − round(K·u*) ≤ e_s^int  (and the
                                                                                    mirror image)
```

The error is then a single rounding of the finished product: $|a_{d,s} - K\cdot\ell_d/c_s| \le 0.5$, i.e. $\le 5\times10^{-7}$ in load units per disk, **independent of c_s**. Summed over a group of even 1 000 disks that is $< 5\times10^{-4}$ — three orders below the solver's mip_gap of 0.02, so it cannot change the selected plan and the two backends stay directly comparable. (C4)/(C5) are already integral in MiB.

Objective coefficients. Same rule — z_d is a constant, so the γ term's coefficient is per-disk and needs no separate γ_scaled:

```
min   Σ_s round(α·W)              · e_s^int                    (imbalance)
    + Σ_d round(β·W·K)            · (1 − x_{d,σ₀(d)})          (number of migrations)
    + Σ_d round(γ·W·K·z_d^TiB)    · (1 − x_{d,σ₀(d)})          (bytes migrated)
    + Σ_v round(κ·W·K)            · (Σ_s y_{v,s} − 1)          (fragmentation)
    + Σ_s round(P·W·K / 2²⁰)      · r_s^MiB                    (reserve violation)
```

The ·K on the count-valued terms puts them on the same footing as α·W·e_s^int, which already carries a factor K inside e_s^int.

Why this matters — the trap in the obvious formulation. Factoring γ out as a standalone per-MiB integer, γ_scaled = round(γ · K / 2^{20}), silently **zeroes the bytes-migrated term at the default weight**:

```
γ = 0.05/TiB,  K = 10⁶   →   round(0.05 · 10⁶ / 2²⁰) = round(0.0477) = 0
```

CP-SAT would then ignore disk size entirely when choosing what to move, diverging from CBC and the heuristic, which use continuous coefficients — and doing so with no error and no warning. Raising K is not a fix worth making: $\gamma\cdot K/2^{20} \ge 0.5$ requires $K \ge 2^{20}/(2\cdot0.05) \approx 1.05\times10^7$, so even $K = 10^7$ still rounds to zero, and the first K that works yields γ_scaled = 1 — a 100 % quantization error on the coefficient. Folding z_d in instead gives, for the §14 fixture's 0.5 TiB disk, round(0.05 · 10^4 · 10^6 · 0.5) = 2.5×10^8: exact, with no minimum-γ restriction at all.

Magnitudes. The largest coefficient is the reserve term, round(P·W·K/2^{20}) ≈ 9.5×10^6 per MiB at P = 1000; a 1 TiB shortfall gives ≈ 10^{13}. The imbalance term reaches α·W·K·Σe_s ≈ 1.5×10^{11} for Σe_s = 15. Both are comfortably inside int64, which is what CP-SAT requires. Assert at model-build time that every coefficient is a non-zero integer wherever its unscaled weight is non-zero — the regression test for the γ trap above — and that the maximum objective magnitude is below 2^{62}.

Warm-start from the current assignment via AddHint(x[d, σ₀(d)], 1), which typically finds the incumbent immediately and spends the rest of the time limit proving the gap. Assert after solving that the

unscaled objective recomputed in floating point agrees with the solver's value to within the rounding bound — a cheap guard against a scaling mistake silently producing wrong plans.

CBC via PuLP. Direct transcription; continuous `e_s`, `Z_s`, `r_s` are fine.

Heuristic fallback (no dependency, and the path for very large groups).

1. **Seed** with the current assignment (not from scratch — we are minimizing *change*).
2. **Repair**: while any `s` violates (C5), move the disk from `s` that most reduces the violation per byte moved, to the feasible storage with the lowest `u_s`.
3. **Descend**: repeatedly evaluate every single-disk move, every pairwise swap, **and every whole-VM co-relocation** (every movable disk of one multi-disk VM moved to the same target storage together, in one trial); apply the one that most improves the full objective (including β, γ, κ); stop when no move improves it or `heuristic_iterations` is reached. The third candidate family was added after the first two (confirmed live on a real production cluster, not found by review): whenever κ is large enough to make moving one disk of an N-disk VM a net loss on its own (it pays κ's fragmentation penalty before a later move could reunite it), neither a single move nor a swap can ever reach the state where relocating the whole VM together is a clear win — descend found *zero* improving moves at all on a real, 196%-imbalanced cluster without this candidate, which a "the path for very large groups" fallback must not do.
4. **Polish**: attempt to reunite fragmented VMs where doing so does not worsen imbalance beyond `imbalance_threshold`. With step 3's whole-VM co-relocation above, this step's remaining scope is narrower than originally written: only an *N-way rotation* across three or more storages (no disk's own move improving alone, and no two disks sharing a VM) is still outside descend's own reach.

Swaps matter and must not be omitted: when both storages are near their capacity limit, no single move is feasible, and only an exchange of two disks can improve the balance.

The heuristic must use the **same** feasibility and objective functions as the MILP path so the two backends are directly comparable; make them shared pure functions.

---

6. Gating — deciding whether to act at all

Applied in order, before the solver runs. Any gate that fails ends the run with "no action".

Drift gate — the "minimum % of changed average traffic" requirement. Compare the current load vector against the one recorded at the last *executed* balance:

```
‖ℓ_now − ℓ_last‖₁ / ‖ℓ_last‖₁   ≥   gates.drift_threshold      (default 0.10)
```

Using the L1 norm over the whole vector, rather than a per-disk test, means many small correlated changes can legitimately trigger a re-plan while one noisy disk cannot.

Vector alignment. The two vectors are indexed over the **union** of disk keys present in either, with a missing disk contributing load 0 to the vector it is absent from. A disk created since the last balance therefore contributes its full current load to the numerator, and a deleted disk contributes its full former load — both are genuine changes to the group's I/O profile and should be able to trigger a re-plan on their own.

Degenerate cases, which must be handled explicitly rather than left to produce a division by zero:

Condition	Behaviour
No `ℓ_last` recorded (first run ever, or state file reset)	**Skip the drift gate**, proceed to the imbalance gate
$\|\|\ell\_last\|\|_1 = 0$ (group was entirely idle) and $\|\|\ell\_now\|\|_1 > 0$	Treat as fully drifted, proceed

Condition	Behaviour
$\|\ell\_last\|_1 = 0$ and $\|\ell\_now\|_1 = 0$	No load, no imbalance — exit “no action”

`ℓ_last` is recorded **only on a run that actually executed at least one migration**, not on every run. Recording it on planning runs would let load creep past the threshold in sub-threshold steps without ever triggering.

Imbalance gate — the “% of IOPS difference over all storages in the group” requirement:

```
(max_s u_s − min_s u_s) / u*   ≥   gates.imbalance_threshold   (default 0.20)
```

Evaluated per group; a group that passes is planned, others are skipped.

Cooldowns — a disk moved within `cooldown_per_disk` (default 24h) is pinned in place; a storage involved in a migration within `cooldown_per_storage` accepts no new incoming moves.

Reserve override — a storage in violation of (C5) bypasses the drift and imbalance gates entirely. Safety is not subject to hysteresis.

---

7. Migration cost and the payback rule

This section implements the requirement that *migrating a very large disk may generate more traffic than it saves*. It is expressed as a **hard acceptance test on the finished plan**, not merely as a soft γ penalty, because a penalty can always be outweighed by a large enough imbalance term.

7.1 Cost

A `move_disk` on a running VM performs a QEMU `drive-mirror`: it reads the whole source volume and writes it to the target, then switches over. For thick provisioning the full provisioned size is transferred. But the mirror is only the first half of the operation — `delete=1` then removes the source volume, and on a storage with `saferemove` enabled that removal is a full-size **zeroing pass**, throttled and often far slower than the mirror it follows:

```
duration_mirror_d = z_d / min(bwlimit, headroom_src, headroom_dst)

duration_wipe_d   = z_d / saferemove_throughput(σ₀(d))     if saferemove is enabled there
                  = 0                                       otherwise

duration_d        = duration_mirror_d + duration_wipe_d

cost_d            = duration_mirror_d · (ω_src + ω_dst)  +  duration_wipe_d · ω_wipe
```

`ω_src` and `ω_dst` (default 1.0 each) are the added in-flight I/O on source and target — a mirror is one sequential reader plus one sequential writer, so 1.0 each is the natural unit and is directly comparable to `ℓ`, which is measured in the same units (§4). `cost_d` is therefore in **load-seconds**. The wipe is charged to the source only, because it is a sequential write over the old volume with nothing happening on the target; `ω_wipe` (`migration.wipe_load_weight`, default 1.0) is its own weight so an operator who knows their array shrugs off a throttled zeroing pass can lower it without touching the mirror weights. §7.3 charges the same quantity to the saturation guard for the whole `draining` window.

Why the wipe term is not a rounding detail. PVE’s LVM `saferemove` (“Wipe Removed Volumes” in the UI) defaults to a throughput of **10 MiB/s**. At that rate the 1.5 TiB disk of the §14 example takes about **44 hours** to wipe, against roughly 2.2 hours to mirror it at 200 MiB/s — the cleanup is twenty times the

move. A cost model that stops at the mirror understates such a migration by that factor and will happily schedule a plan that occupies the source array for two days.

Read `saferemove` and `saferemove_throughput` from `GET /storage` (§3.5 — the list form, not `GET /storage/{id}`) per storage; never assume. `migration.account_saferemove_wipe: false` disables the term for an operator who has verified their storages do not wipe, but the default is to account for it. Note the knock-on effects, all covered in §9.3: the wipe also determines when the source's space is actually released, and it holds a storage-level lock while it runs.

7.2 Benefit

The plan reduces the imbalance objective from `E_before = Σ_s e_s` to `E_after`. That reduction persists until the workload changes, which we bound by the payback horizon `H`:

```
benefit  =  (E_before − E_after) · H
```

also in load-seconds. Both sides of the comparison are thus in the same unit, which is the whole point of using I/O time as the load metric.

7.3 Acceptance

```
accept plan   ⟺   benefit  ≥  migration.payback_ratio · Σ_d cost_d
```

with `H = 7d` and `λ = 10` by default. Additional **hard** rules, applied per move, that reject individual migrations regardless of the aggregate test:

- `duration_d > migration.max_single_move_duration` (default 6h) → reject the move;
- the move would push either endpoint above `migration.saturation_ceiling · saturation_load` during the mirror → defer the move to a later run rather than reject the plan (see below); skipped for a storage with no `saturation_load` configured;
- the move violates the transient reserve invariant of section 8 → reject.

Defining "during the mirror". `u_s` as used everywhere else is a p95 over the lookback window — a robust *statistic*, not an instantaneous reading — so adding an instantaneous `ω` to it would mix two different kinds of quantity. Define the check explicitly:

```
L_during(s)  =  L̂_s(duration_d)  +  Σ_{m in flight at s} ω_role(m,s)

check:  L_during(s)  ≤  saturation_ceiling · N_s        for s ∈ {src, dst}
```

where `L̂_s(Δ)` is the **forecaster's upper bound on `L_s`** — the storage's *aggregate* load over a horizon equal to the move's expected duration (§10), in average in-flight I/O requests, the same units as ℓ (§4) and as `ω`. It is **not** the capability-normalized `u_s`; mixing the two here was the original defect in this rule.

`ω_role(m,s)` depends on the move's *state*, not only on its endpoints. A move stays in the in-flight set `M` until it reaches `done` (§8.2). That is right for capacity, but a *fixed* role charge would be wrong for load: once the mirror has switched over, this move writes nothing more to the target, while the source is being **zeroed** for as long as `saferemove` takes. So:

state	charge on src(m)	charge on dst(m)
mirroring	ω_src	ω_dst
draining	ω_wipe	0
done	0	0

`ω_wipe` is `migration.wipe_load_weight`, default 1.0 — the zeroing pass is one sequential writer, so 1.0 is the natural value and it is the same quantity §7.1 charges for `duration_wipe_d`. With this, a 44-hour wipe

on a busy source stays visible to the saturation guard for its whole duration instead of vanishing from the check the moment `move_disk` reports OK, which is the only way the guard can protect the *next* move scheduled onto that storage. The capacity invariant of §8.1 needs no change: it already holds the move in `M` until `done`, and the source-side byte accounting of `mirroring` and `draining` is identical.

Two honest caveats. The wipe is throttled by construction (10 MiB/s by default), so charging it a full ω is conservative — deliberately so, because the alternative is to under-count a storage that is busy zeroing 1.5 TiB. And this remains a best-effort guard: a deployment with no `saturation_load` set skips it entirely, and there `execution.cooldown_per_storage`, sized against the wipe time (§9.3), is the blunter mitigation that still works.

`N_s` is what the check is measured against, and it is not `c_s`. `c_s` is a *relative* capability weight whose default is 1.0 and whose absolute value is meaningless — only the ratios between storages in a group affect the balance objective, so `saturation_ceiling · c_s` compares a physical queue depth against a dimensionless preference. That is dimensionally wrong in both directions: with `c_s = 1.0` a storage carrying an entirely healthy `L_s = 6.5` would fail a 0.85 ceiling outright, and a storage weighted `c_s = 0.5` would be held to half the ceiling of its peer purely for being labelled less capable. Instead:

```
N_s = storages[].saturation_load     — the number of concurrent I/O requests storage s services
                                       before queueing delay dominates. An absolute, physical
                                       property of the array (roughly its effective queue depth).
```

`N_s` has **no safe default and is `null` unless the operator sets it**, in which case the check is skipped for that storage and `pve-storage-drs explain` says so. We cannot infer it: the observed peak `L_s` is not a capacity (an idle storage would get a tiny `N_s` and reject every migration onto it, which is exactly backwards), and neither `c_s` nor the LUN size tells us anything about queue depth. Obtain it from the array's documented queue depth, or empirically as the `L_s` at which measured latency starts climbing super-linearly. Sizing `N_s` in the same units as ℓ is straightforward because both come from the same Little's-law quantity.

This is deliberately a **best-effort guard**, not a physical limit: even with `N_s` set we have no model of the array's true saturation point, only the load we can attribute to guests. `max_single_move_duration` and the transient reserve invariant of §8.1 are the hard bounds and are always active; this one exists to avoid the obviously bad case of starting a long mirror onto a storage that is already close to its service limit. A deployment that leaves every `saturation_load` unset is fully supported and loses only this one advisory check.

If the plan fails the aggregate test, re-solve with β and γ doubled and retry, up to three times. This naturally converges on the smaller subset of high-value moves rather than abandoning the run — usually the one or two disks with the highest ℓ`_d / z_d` ratio, which is exactly the right thing to move.

ℓ`_d / z_d` — load per byte — is worth surfacing in `pve-storage-drs explain` output. It is the single best indicator of a good migration candidate: high I/O concentrated in a small disk.

---

8. Migration ordering

The solver produces a *target assignment*. It says nothing about the order of moves, and order matters: the requirement that the snapshot reserve holds **even during storage migrations** is a constraint on every intermediate state, not just the endpoints.

8.1 The transient invariant

During a `move_disk` of disk `d` from `a` to `b`, the volume exists on **both** storages — the mirror target is fully allocated before the switchover, and the source is only removed afterwards by `delete=1`. So while a single move is in flight, `b` must satisfy:

```
used_b + z_d + f_b · max(Z_b, z_d)   ≤   C_b
```

Note the `max(Z_b, z_d)`: if the incoming disk is the new largest on `b`, the required reserve grows at the same moment the disk arrives. This is the case most likely to be missed, and the one most likely to fill a SAN LUN.

The source `a` gets no relief until the move completes, so a plan that depends on freeing space on `a` to make room on `a` is simply infeasible and must be ordered around.

Generalized to concurrent moves. `execution.max_concurrent_migrations` may exceed `1`, and then the single-move form above is **not sufficient**: several disks can be landing on `b` at once, and none of their sources release space until each completes. For an in-flight set `M`, every storage `b` must satisfy:

```
used_b  +  Σ_{m∈M : dst(m)=b} z_{disk(m)}
        +  f_b · max( Z_b , max_{m∈M : dst(m)=b} z_{disk(m)} )   ≤   C_b
```

Both the sum and the inner `max` are over the same in-flight set. Implement this as the single feasibility predicate and call it with `M = {m}` for the sequential case, so there is only one version of this rule in the codebase.

`concurrency_ok(state, m)` is then defined as: adding `m` to the current in-flight set

1. keeps the generalized invariant above satisfied on **every** storage;
2. keeps `|M| ≤ max_concurrent_migrations`;
3. keeps the count of in-flight moves touching any single storage — **as either source or target** — at or below `max_concurrent_per_storage`;
4. respects the saturation check of §7.3, which sums `ω_role(m,s)` over **every** move still in `M` at that storage — including moves in `draining`, whose source is charged `ω_wipe` and whose target is charged nothing;
5. violates no per-disk or per-storage cooldown.

With the default `max_concurrent_per_storage: 1`, two moves targeting the same storage serialize automatically and the generalized form collapses to the single-move form. That is the recommended configuration, and raising it should be a deliberate act on a storage with ample headroom.

8.2 Scheduling algorithm

```
pending ← { moves implied by the target assignment }
state   ← current allocation
order   ← []

while pending:
    feasible ← { m ∈ pending : transient_invariant_ok(state, m)
                               and concurrency_ok(state, m)
                               and not in cooldown }
    if feasible is empty:
        if a staging move exists:  order.append(staging_move); continue
        else: report deadlock with the blocking storages; break

    m ← argmax over feasible of  (imbalance reduction) / cost_m
    order.append(m)
    state ← apply(state, m)          # target charged immediately; the source is charged
                                     # until its volume is observed gone (see below)
```

Ordering by **imbalance reduction per unit cost** means the plan front-loads its value: if the operator aborts halfway, or a maintenance window closes, the moves that mattered most have already run. Two exceptions take priority and are scheduled first regardless of ratio:

1. moves that resolve a storage currently violating (C5);
2. moves that *free* space on a storage which some later move needs.

The source is not freed when the task succeeds. `apply(state, m)` must not optimistically credit the source with `z_d` bytes back. With `saferemove` on the source storage the old volume still exists — fully allocated — for the whole duration of the zeroing pass (§7.1), which can be far longer than the move itself. A move therefore leaves the in-flight set `M` in two stages:

```
mirroring  →  draining  →  done

mirroring:  target charged z_d, source still charged z_d   (the drive-mirror window of §8.1)
draining:   target charged z_d, source still charged z_d   (the move_disk task has reported OK,
                                                            but the source volume is still there)
done:       source volume absent from /storage/{a}/content
```

For the generalized transient invariant of §8.1 the *source-side* accounting of `mirroring` and `draining` is identical, so keep a move in `M` until it reaches `done` and the invariant needs no change at all. The *load* charge is not identical across the two states, and §7.3's `ω_role(m,s)` table is what distinguishes them: a draining move charges `ω_wipe` to its source and nothing to its target. Keeping the move in `M` is therefore what makes the wipe visible to the saturation guard as well as to the capacity check — which is the point, since the wipe is by far the longer of the two windows on a large disk. What does change is that ordering rule 2 — "moves that free space a later move needs" — cannot be satisfied within a run when the source wipes slowly. The scheduler must therefore treat a predicted free-space release as **unrealised until observed**, and a plan whose feasibility depends on one is split rather than executed on faith (§8.3, option 2).

8.3 Deadlock and staging

Three storages each near capacity can produce a cycle in which no move is individually feasible though the target assignment is perfectly valid — the classic pebble-motion deadlock. Resolution, in order of preference:

1. **Staging move** — find any storage in the group with enough slack to temporarily accept the smallest disk in the cycle, move it there, complete the cycle, then move it to its target. Costs one extra migration; permit it only if the *whole* plan still passes the payback test with that extra cost included.
2. **Split the plan** — execute the feasible subset now, re-plan on the next run. Often the workload has changed anyway.
3. **Report** — emit the blocking set and stop. Never force a move that breaches the reserve.

Guard staging moves against loops: a disk may be staged at most once per plan, and a staged disk is exempt from the per-disk cooldown for the duration of that plan only.

---

9. Execution

9.1 Modes

Mode	Behaviour
`dry-run` *(default)*	Print the plan, the before/after balance, the payback arithmetic, and the exact API calls that *would* be issued. Change nothing.
`confirm`	Execute the plan one move at a time, prompting before each. Re-validate the transient invariant immediately before each prompt, since the cluster may have changed. Offer `[y]es / [n]o skip / [a]ll remaining / [q]uit`.

Mode	Behaviour
`auto`	Execute unattended, subject to concurrency caps and `execution.time_windows`.

`auto` must additionally: refuse to start a move that cannot finish inside the remaining time window; stop cleanly at window close rather than aborting an in-flight move; and honour `max_migrations_per_run`.

9.2 Performing one move

```
POST /nodes/{node}/qemu/{vmid}/move_disk
     disk={device} storage={target} delete=1 bwlimit={KiB/s}
  → UPID
poll GET /nodes/{node}/tasks/{upid}/status every execution.poll_interval_seconds
  until status == "stopped"; task success ⟺ exitstatus == "OK"
```

Task success is **not** the completion criterion — see §9.3. The move is done only once the source volume has actually disappeared and the VM's lock has cleared.

`bwlimit` is **KiB/s** in the API, while `migration.bwlimit_bytes_per_sec` is bytes/s; convert at the call site and nowhere else.

With `max_concurrent_migrations > 1` this loop launches up to the cap (subject to `concurrency_ok`, §8.1) and polls **all** in-flight UPIDs each cycle, starting the next queued move as each slot frees. With the default cap of 1 it degenerates to the sequential form above.

Before **every** move, re-read the live state rather than trusting the plan:

1. re-fetch `/nodes/{node}/qemu/{vmid}/config` and confirm the disk is still on the expected source — the VM may have been touched by an operator, or live-migrated to another node by the PVE 9.2 Dynamic Load Balancer (§1), changing `{node}`;
2. re-fetch `/nodes/{node}/storage/{target}/status` and re-check the transient invariant against *actual* current free space;
3. confirm the VM is still running and untagged for exclusion;
4. confirm `config.lock` is empty — if not, wait per §9.3 rather than failing;
5. confirm no snapshot has appeared for the VM since planning (§3.7); if one has, drop the move and re-plan — `delete=1` would be rejected by PVE anyway.

These re-reads bypass the per-run topology cache (§3.5) for this VM and this storage only. Steps 4 and 5 are cheap: both come from the same `/qemu/{vmid}/config` response as step 1.

Re-plan protocol. A mismatch is a *normal* outcome in a live cluster, not an error, and must not be allowed to loop:

1. Abandon the remaining moves in the current plan. Do not attempt to patch it — the state it was computed against no longer holds.
2. Record the moves already completed in `state.json` (including their cooldown timestamps) so the next pass sees them as history rather than re-deriving them.
3. Re-invoke the **whole** pipeline from the new observed state: gates, load model, solver, payback, ordering. The gates may well conclude no further action is needed, which is a correct outcome.
4. Cap re-plans at `execution.max_replans_per_run` (default 3). On exceeding it, stop and report — a cluster churning faster than the engine can plan is a condition for a human to look at, not to iterate against.
5. In `auto` mode, a re-plan inherits the remaining time window; if too little remains for the cheapest queued move, stop cleanly rather than starting one that cannot finish.

9.3 Locks, and why a completed move is not a finished move

Two distinct mechanisms can make a VM untouchable, and the engine must handle both. Neither is an error condition — both are ordinary states in a working cluster — so the response to both is to **wait**, not to fail.

1. The VM config lock. PVE writes a `lock:` line into the VM config for any operation that must not be interrupted. Its documented values are `backup`, `clone`, `create`, `migrate`, `rollback`, `snapshot`, `snapshot-delete`, `suspending` and `suspended`. Any of them makes `move_disk` fail with *"VM is locked"*.

```
before issuing move_disk for VM v:
    lock ← config(v).lock                    # /qemu/{vmid}/config, or /status/current
    while lock is set:
        if waited > execution.locks.wait_timeout:  → apply execution.locks.on_timeout
        sleep execution.locks.poll_interval
        re-read lock
```

Treat the value set as **open-ended**. Never whitelist "harmless" locks and proceed anyway: a future PVE version may add a value, and the failure mode of guessing wrong is a half-completed operation on someone else's backup. Any non-empty `lock` means wait. Log which lock was seen and for how long — a VM stuck in `backup` for six hours is something the operator wants to know about, and it is the main reason the wait timeout is measured in hours rather than minutes.

The pre-flight check is also a *planning*-time input: a VM locked when the run starts is pinned by (C2) for that run, so the solver does not build a plan around a disk it cannot touch. The check here is the second line of defence, because a lock can appear between planning and execution.

2. The post-move wipe — the one that is easy to miss. When `move_disk delete=1` completes and the task reports `exitstatus: OK`, the migration is *not* over on a storage with `saferemove` enabled. PVE then zeroes the old volume at `saferemove_throughput` (default **10 MiB/s**, §7.1), which for a large disk runs for hours or days. During that window:

- the source volume still exists and its space is **not** reclaimed — §8.2's `draining` state;
- the operation holds a storage-level lock, so other volume operations on that storage queue behind it, and a subsequent `move_disk` touching the same VM or storage can fail even though *our* move finished cleanly;
- the task list shows nothing running, so a naive "poll until the UPID stops" loop concludes the move is done and immediately issues the next one — straight into the lock.

This is why the executor's completion criterion is deliberately stronger than task success:

```
move m from a to b is DONE  ⟺  task(upid) exitstatus == OK
                            ∧  volume(m) absent from GET /storage/{a}/content
                            ∧  config(vmid(m)).lock is empty
```

Poll all three at `execution.poll_interval_seconds`, bounded by `execution.source_release.timeout` (default **48h**, sized for a multi-TiB wipe at 10 MiB/s). On timeout, do not fail the run: mark the storage `draining`, exclude it as both source and target for the remainder of the run, report it, and let the next run re-evaluate from observed reality. Set `execution.source_release.wait: false` only on storages verified not to wipe — with `saferemove` off, the volume disappears immediately and this condition costs one extra API call.

Sizing the knobs against each other. With saferemove at its default throughput, one move can occupy its source storage for far longer than a whole planning cycle. Two consequences the implementer should not have to rediscover:

- `gates.cooldown_per_storage` must exceed the expected wipe time for that storage's largest disk, or the next run will plan moves onto a storage that is still draining and stall in `9.3`'s wait loop. `pve-storage-drs verify-storages` computes `z_max / saferemove_throughput` per storage and warns when the configured cooldown is shorter, or when it exceeds `migration.max_single_move_duration`.
- Keep `execution.max_concurrent_per_storage: 1`. Two moves off the same source mean two concurrent wipes sharing one throttle, so both take twice as long while both sources stay fully allocated.

9.4 Failure handling

`move_disk` is atomic from the caller's perspective: on failure QEMU cancels the mirror and the source volume remains authoritative, so there is nothing to roll back. The realistic hazards are:

- **Orphaned target volume.** A failed or cancelled mirror can leave `vm-101-disk-0` on the target. After any failure, list `/nodes/{node}/storage/{target}/content` and warn about volumes owned by the VM that are not referenced in its config. Do **not** delete them automatically — report them. They count against the reserve until removed, so surfacing them matters.
- **Partial plan.** With `abort_on_failure: true` (default), stop and report; the cluster is in a valid intermediate state and the next run will re-plan from reality.
- **Task supervision loss.** If the engine dies mid-move, the PVE task continues. On startup, check for running `move_disk` UPIDs owned by the DRS user before planning anything.

9.5 Output

Every mode emits the same machine-readable plan (JSON) plus a human summary. The example below is the section 14 fixture at `beta_move_count: 0.5` (the two-move variant):

```
Group fc-tier1 — imbalance 255% (threshold 20%) → ACT
  san-a  u=6.50  ████████████████████  used 4.5/8.0 TiB  ⚠ reserve short by 0.5 TiB
  san-b  u=0.70  ██                    used 1.5/8.0 TiB
  san-c  u=0.20  █                     used 0.5/8.0 TiB

  1. 102:scsi0  san-a → san-c   1.5 TiB   ~2.2h   Δimbalance −4.53   ℓ/z 1.67
  2. 101:scsi1  san-a → san-b   1.0 TiB   ~1.5h   Δimbalance −2.00   ℓ/z 1.00

  after: san-a u=3.00  san-b u=1.70  san-c u=2.70   spread 53% (from 255%)
  payback: benefit 3.95e6 load·s vs cost 2.62e4 load·s → ratio 151 (need 10) ✓

  pinned (not movable this run):
    106  snapshots present (2)      1.0 TiB  ℓ 0.9  on san-a  → clear snapshots to unblock
    107  locked: backup             0.5 TiB  ℓ 0.3  on san-b  → waited 0s, re-check next run
    109  excluded by tag no-drs     0.2 TiB  ℓ 0.0  on san-c
  pinned load 1.2 of 8.6 (14%, warn at 25%);  best achievable spread given pins: 44.6%
```

The pinned block is not optional decoration — it is the "complain" half of the skip-and-complain policy of §3.7, and it is the only place an operator learns which snapshots to clear.

As built (REVIEW.md V-02): `plan`/`apply` print none of the above beyond the move list and payback verdict — neither renderer carries a pinned field, a fragmentation line, or a pinned-load line. That narration lives in `pve-storage-drs explain` instead (`docs/manual/29-explain.md`'s `pinned (not movable this run):/cannot fully consolidate:/pinned load ...` sections, including the per-pin → action hints shown above); `show-load` also names each pin inline, per disk, as `[pinned: <reason>]`. Read every "pinned block"/"plan output" reference above as `explain`'s output, not `plan`'s or `apply`'s.

---

10. Forecasting

The default decision statistic is the p95 of the trailing window, which is deliberately conservative and needs no model fitting. Forecasting is behind an interface so it can be strengthened without touching the optimizer:

```python
class Forecaster(Protocol):
    def required_range(self) -> timedelta:
        """How much history this model needs. metrics.py serves exactly this."""

    def predict(self, series: TimeSeries, horizon: timedelta) -> Forecast:
        """Returns point estimate and an upper bound for the horizon."""
```

10.1 Each forecaster owns its data range

`window.lookback` is the **decision** window — the period whose load we are balancing. It is *not* the amount of history a forecaster needs, and conflating the two makes the seasonal models unreachable: Holt-Winters with `seasonal_periods = 288` (24 h at a 5 m step) needs `2 × 288 = 576` samples, i.e. **48 h**, which a 24 h window can never supply. It would silently fall back to `quantile` forever.

So `metrics.py` must expose `query_range` over an **arbitrary** range, not just `window.lookback`, and each forecaster declares what it needs:

Implementation	`required_range()`	Point estimate	Upper bound
`quantile` *(default)*	`window.lookback` (24 h)	p95 over `W`	`quantile_over_time(upper_quantile)`, default p99
`seasonal_naive`	`max(lookback, seasonal_lookback_days)` (7 d)	median across same-hour-of-day samples	p95 across those samples
`holt_winters`	`max(lookback, 2 · seasonal_periods · step)` (48 h)	fitted forecast at `horizon`	point + `z`·σ of in-sample residuals, `z = 2`

The optimizer consumes the **upper bound**, never the point estimate. Being wrong in the direction of "this disk is busier than it looks" costs a slightly suboptimal balance; being wrong the other way migrates a disk onto a storage that is about to be saturated. For the default `quantile` forecaster this makes the distinction concrete rather than vacuous: the point estimate is `window.quantile` (p95) and the bound is `window.upper_quantile` (p99), so the optimizer sees p99.

As built (REVIEW.md T-03): the decision statistic that actually drives the gates, solver, payback and ordering is `window.quantile` (the point estimate), computed once per group by `loadmodel.compute_group_load()`. The upper bound described above is real and exercised, but its only consumer is §7.3's saturation-ceiling guard, and only for a storage that configures `saturation_load` — the guard calls a `Forecaster` (built here, gated by the §10.2 backtest below) to get $\hat{L}\_s(\Delta)$. Wiring the upper bound into the optimizer's own input, as this section describes, remains future work; until then, treat every occurrence of "the optimizer consumes the upper bound" in this document as the target design, not the current behaviour.

Forecasts are produced **per disk**. Where §7.3 needs a per-*storage* bound $\hat{L}\_s(\Delta)$, it is the sum of the per-disk upper bounds over the disks assigned to `s` in the state being evaluated: $\hat{L}\_s(\Delta) = \Sigma\_{d : x\_{d,s}=1}\ \hat{u}\_d(\Delta)$. Summing upper bounds is conservative — it assumes the disks peak together — which is the right direction for a guard whose failure mode is starting a mirror onto an already-busy array.

Config validation (§11.1) must **reject** a configuration whose Prometheus retention or whose selected forecaster and window are mutually inconsistent, rather than silently degrading. Enabling a seasonal model is a statement that the history exists to support it.

10.2 Implementation warnings

- **Do not compute Holt-Winters in PromQL.** Prometheus's `holt_winters` was renamed `double_exponential_smoothing` in Prometheus 3.x and requires `--enable-feature=promql-experimental-functions`. More importantly, despite the historical name it is **double** exponential smoothing — level and trend only, with **no seasonal component** — so it cannot learn a daily cycle. Seasonal forecasting must happen engine-side on data pulled via `query_range`.
- Require at least `2 × seasonal_periods` samples before trusting a Holt-Winters fit, and fall back to `quantile` otherwise — with a **logged warning**, since a silent fallback hides a misconfiguration.
- Validate by backtesting: fit on `[t−2T, t−T]`, predict `[t−T, t]`, compare against actual. Refuse to let a model whose backtest error exceeds the imbalance threshold drive migrations.

---

11. Configuration

Where it lives: `/etc/pve/drs.yaml`. That path is on pmxcfs, the cluster filesystem, so the file is replicated to every node automatically: one edit, one config, no per-node drift, and no question about which copy is authoritative. It is also an ordinary path, so a management host that is not a cluster member can simply provide it at the same location and every command, example and manpage stays true on both kinds of host.

Resolution order, first match wins:

#	Source	On failure
1	`--config PATH` (`-c`)	Error and exit non-zero
2	`$PVE_STORAGE_DRS_CONFIG`	Error and exit non-zero
3	`/etc/pve/drs.yaml`	Error naming the path and pointing at the shipped example

An **explicitly requested** config that is missing, unreadable or invalid is a hard failure. Never fall through to the default: a run that silently balanced a production cluster from a different file than the operator named is the worst outcome in this document. Every run logs the resolved path and the SHA-256 of the file it actually read, in the first line of output and in the JSON report.

Three consequences of living on pmxcfs, all of which the implementation must respect:

1. **An edit is cluster-wide and immediate.** There is no staging step and no per-node rollout. A typo is live everywhere the moment it is saved, which is why §11.1's validation is a hard failure rather than a warning and why `dry-run` is the default mode.
2. **Do not put the state file there.** `state.json` is written on every run; pmxcfs is a small, quorum-gated, cluster-replicated store meant for configuration. Reads do not need quorum, so a node that has lost quorum can still read its config and plan, but writes do — keep `state.path` on local disk (§11.2).
3. **Treat the file as readable by the web server.** Files under `/etc/pve` carry group ownership `www-data` by default, which puts a plaintext `password:` within reach of anything running as the PVE web server. Prefer an API token restricted to the privileges of §3.5, and keep the secret out of the file entirely using `PVE_PASSWORD` / `PVE_TOKEN_SECRET` from the environment or a systemd credential. **Verify the ownership and mode on the target cluster** (`ls -l /etc/pve/drs.yaml`) before storing any secret in it — this document does not assume it.

A shared config does not make the tool cluster-aware. The obvious next step after putting the config on pmxcfs is to enable the systemd timer on every node, and that is wrong: `state.json` is node-local, so

each node keeps its own lock, its own cooldowns and its own drift baseline, and the `fcntl` lock of §11.2 cannot see the other nodes at all. What does cross the cluster is the startup scan for in-flight `move_disk` UPIDs owned by the DRS user (§13) — it is the only reason two concurrent instances degrade to "slow and redundant" instead of "conflicting". **Run the timer on exactly one host.**

See `config/drs.example.yaml` for the fully annotated reference. The requirement-to-setting mapping:

Requirement	Setting
Storage groups VMs may not leave	`groups[].storages[]` — literal ids or `/regex/` patterns (§11.4)
2× largest disk free for snapshots	`snapshot_reserve.factor` (default `2.0`), per-storage override
Min % changed traffic before migrating	`gates.drift_threshold` (default `0.10`)
% I/O difference across the group	`gates.imbalance_threshold`
Timeframe considered	`window.lookback` (default `24h`)
Minimal number of migrations	`objective.beta_move_count`
Keep a VM's disks together	`objective.kappa_vm_affinity`
Migration load accounted for	`migration.*`, `objective.gamma_move_bytes_per_tib`
Manual vs automatic	`execution.mode`
Forecasting	`forecast.model`

The config carries `schema_version: 1` at the top level. `config.py` rejects an unknown **major** version with an explicit message naming the version it understands, so a future incompatible change has a migration path instead of misinterpreting fields.

11.1 Validation rules

Validate with `jsonschema` for structure, then apply these semantic rules. Every one of them is a failure that produces a clear error and a non-zero exit, never a warning-and-continue — a misconfigured balancer moving production disks is worse than one that refuses to start.

Rule	Rationale
`schema_version` major matches	Forward compatibility
Every group non-empty, ≥ 2 storages **after pattern expansion** (§11.4)	A one-storage group has nothing to balance; with patterns the count that matters is storages matched, not entries written
No storage appears in two groups, after pattern expansion	A disk's group would be ambiguous; a pattern matching a storage another group also names is the same defect
Storage ids exist in the cluster; every `/…/` pattern matches at least one storage (§11.4)	Catches typos before they silently exclude disks or silently shrink a group
Every `/…/` pattern compiles as a Python regular expression, checked at load time	A malformed pattern must fail with the compiler's own message, not crash at match time (§11.4)
Within a group, no storage is matched by two pattern entries	Which entry's options apply would be arbitrary; a literal entry overriding a pattern is allowed and is not this error (§11.4)
`capability_weight > 0`	Appears in a denominator
`reserve_factor ≥ 0`, `min_free_bytes ≥ 0`	Negative reserve is meaningless
`0 ≤ drift_threshold ≤ 1`, `0 ≤ imbalance_threshold ≤ 1`	They are ratios
`quantile ∈ (0,1)`, `upper_quantile ∈ (0,1)`, `upper_quantile ≥ quantile`	The bound must not sit below the point estimate
`min_coverage ∈ (0,1]`	A ratio; 0 would accept a disk with no data
Metric names non-empty; label names non-empty and pairwise distinct	A duplicated label name silently collapses series
`rate_window ≥ 4 × metrics.pvestatd_push_interval`	Below this, `rate()` sees too few points. The interval is a PVE-side setting the tool cannot read, so it is declared in config (default `60s`, PVE's own default) and `verify-metrics` cross-checks it against the observed sample spacing of a live series, erroring if the two disagree by more than 20%
`window.lookback ≥ forecaster.required_range()`	See §10.1 — otherwise the model can never run
`payback_ratio > 0`, `payback_horizon > 0`	Zero disables the safety test
`saturation_ceiling ∈ (0,1]`	A fraction of `saturation_load`, not of `capability_weight`

Rule	Rationale
`saturation_load > 0` where set; warn once per run for each storage where it is unset	§7.3's guard is silently inactive without it
`max_concurrent_* ≥ 1`	Zero would deadlock the scheduler
`execution.locks.wait_timeout > 0`, `on_timeout ∈ {skip, abort}`	A zero timeout turns every ordinary backup window into a failed run
`execution.source_release.timeout ≥ z_max / saferemove_throughput` for every storage where saferemove is on	Otherwise every large move times out into `draining` (§9.3)
`gates.cooldown_per_storage ≥ z_max / saferemove_throughput` (warn, not error)	The next run would plan onto a still-draining storage
`report.warn_pinned_load_fraction ∈ (0,1]`	A ratio
Time windows: `start ≠ end`; crossing midnight allowed and explicit	Ambiguity here silently disables `auto`
`execution.mode ∈ {dry-run, confirm, auto}`	Typo must not silently fall back to acting

11.2 `state.json`

The only persistent state, at `state.path`, default `/var/lib/pve-storage-drs/state.json`. Local disk, one copy per host, deliberately **not** on `/etc/pve` for the reasons in §11. Small, versioned, and written atomically (temp file + `os.replace`):

```
{
  "schema_version": 1,
  "lock": { "pid": 12345, "host": "mgmt01", "acquired_at": "2026-09-04T02:00:00Z" },
  "last_balance": {
    "at": "2026-09-03T22:14:03Z",
    "load_vector": { "fc-tier1:101:scsi0": 3.0, "fc-tier1:102:scsi0": 2.5 }
  },
  "cooldowns": {
    "disk":    { "fc-tier1:101:scsi1": "2026-09-03T22:41:55Z" },
    "storage": { "fc-tier1:san-b":     "2026-09-03T22:41:55Z" }
  },
  "inflight_upids": [],
  "staged_disks": []
}
```

- Keys are `"<group>:<vmid>:<device>"` and `"<group>:<storage>"`, so a vmid reused after a VM is destroyed and recreated in a different group cannot collide.
- `last_balance` is updated **only after a run that executed at least one migration** (§6).
- **Locking:** `fcntl.LOCK_EX` on the file for the duration of a run. If the lock is held but `lock.pid` is not alive on `lock.host`, reclaim it and log the reclamation — a killed run must not block the timer forever. A live PID means another instance is running: exit 0 quietly.
- `inflight_upids` is written *before* issuing each `move_disk` and cleared on completion, so a crashed run can be reconciled on the next start (§13).
- Losing this file is safe but not free: cooldowns and drift history reset, so the next run may migrate sooner than intended. Treat it as state to back up, not as a cache.

11.3 Global command-line options

Accepted before the subcommand, and shown by `pve-storage-drs --help` with their defaults:

Option	Default	Effect
`-c, --config PATH`	`/etc/pve/drs.yaml`	Read the configuration from `PATH`. Missing or invalid is a hard failure (§11)

Option	Default	Effect
`--group NAME`	all groups	Restrict the run to one group; repeatable. Groups are independent (§5), so this changes nothing about the result for the groups selected
`--mode {dry-run,confirm,auto}`	`execution.mode`	Override the execution mode for this run only
`--json`	off	Emit the machine-readable report of §9.5 instead of the human one
`-v, --verbose`	normal	More detail on stderr. Repeatable
`--quiet`	normal	Warnings and errors only, for the systemd timer
`--version`	—	Version, then exit
`--manual`	—	Show `pve-storage-drs(1)` (§8.5 of `AGENTS.md`)

Two rules the implementation must honour.

Every `--mode` override is logged, and an override toward less safety is a warning. Order the modes `dry-run` < `confirm` < `auto`. Moving *down* that order — `auto` to `confirm`, `confirm` to `dry-run` — is logged at info: an operator being more careful than their config needs no ceremony. Moving *up* it is the operator deliberately removing a barrier they themselves configured, whether that barrier is the dry run or the per-step confirmation, so `dry-run → confirm`, `dry-run → auto` and `confirm → auto` all log at **warning** level, naming both values. In `auto` mode the log is the only record a human will see (§2.1), and "why did it move disks when the config said confirm" must be answerable from it.

No option may set a value that `config.py` would have rejected in the file. The command line goes through the same validation (§11.1), because a knob that is only checked on one of its two paths is a knob that is not checked.

11.4 Storage name patterns in `groups[].storages[]`

A `storages[]` entry may name storages by regular expression rather than by literal id: an `id` value that both begins and ends with `/` is a pattern, and the text between the slashes must compile as a Python `re` expression. `/san-.*/` is a pattern; `san-a` stays a literal id and behaves exactly as before. The form is additive — no `schema_version` bump — and it cannot collide with a literal one: PVE storage IDs never contain `/`, so a value wrapped in slashes can only have been meant as a pattern.

```yaml
groups:
  - name: fc-tier1
    storages:
      - id: /san-.*/            # pattern: every san-* LUN, present and future
        capability_weight: 1.0   # the entry's options apply to EVERY storage it matches
      - id: san-b               # literal: takes precedence over the pattern above
        capability_weight: 0.5  # the documented exception for one slower array
```

Four rules, each shaped so that the one new failure mode — a pattern that matches too much or nothing at all — stays checkable instead of silent:

- **Matching is `re.fullmatch`, case-sensitive.** The pattern must match the *entire* storage id. `/prod/` names exactly the storage `prod`; under substring semantics it would also capture `preprod`, which is the classic too-greedy pattern and the reason whole-id matching is the rule. Python `re` syntax throughout: inline flags such as `(?i)` work, and there is no flag suffix after the closing slash.
- **The entry's options apply to every storage it matches.** A pattern entry accepts the same per-storage options as a literal one (`capability_weight`, `reserve_factor`, `saturation_load`), and every

matched storage inherits them. This is the point of the feature: one entry weights or reserves a whole LUN family.

- **A literal entry beats a pattern.** Within one group, a storage named by a literal entry uses the literal's options even when a pattern also matches it — pattern as the default, literal as the exception, and deliberately not an error, because without this rule a catch-all pattern would make it impossible to single out one member of its own group. Two *patterns* matching the same storage in one group are a hard error (§11.1): which entry's options should apply would be arbitrary, and the config refuses to guess. Precedence is semantic, not positional — entry order never matters.
- **Across groups, disjointness is checked on the expanded set.** A storage matched — literally or by pattern — by entries of two different groups is a hard error naming the storage and both entries. A disk's group must stay unambiguous, exactly as §11.1 already demands for literals.

Expansion happens at run start, against the live cluster. Every pattern is matched once per run against the storage definitions (`GET /storage`, §3.5 — the same call that already validates a literal id's existence and content type, so pattern and literal ids are checked against one consistent source), in the same step that checks literal ids for existence, and the result is part of the per-run topology cache. A storage a pattern matches that has a definition but is reported active by no node (disabled) is also a hard error, naming the pattern and the storage — the same fail-safe outcome a literal reference to it would hit, but with a message that does not require the operator to already know a pattern was involved. Which storages a pattern matches is a property of the cluster, not of the file: a LUN added to the cluster joins its group on the next run with no config edit — half the reason to use a pattern — and a pattern that matches nothing is handled exactly like a literal id that does not exist, a hard error before anything is planned, because a typo is the likelier cause and a silently shrunken group the likelier consequence. Nothing downstream ever sees a pattern: `S` (§5.1), the (C2) eligibility pass, cooldown keys and the `state.json` keys of §11.2 all carry real, expanded storage ids. Those keys embed the group name, so a later config change that moves a storage into a different group leaves its old cooldown keys as stale entries that simply stop matching — harmless. Group names themselves, `--group` (§11.3) and the `exclude.*` lists stay literal; only `storages[]` entries accept the `/…/` form.

Validation and visibility. `config.py` compiles every pattern at load time and reports a malformed one with the compiler's own message; the zero-match and overlap rules run every run once the inventory is loaded, because they compare the file against the cluster. Because an over-broad pattern is the one new way to misconfigure a group, the expansion is always visible: every run logs `entry → matched ids` at INFO, and `pve-storage-drs verify-storages` (§3.5) prints the expansion next to each storage's properties and lists cluster storages matched by no group — the same "ungrouped, not managed" visibility (C2) gives `show-load`. One consequence to keep in mind when writing a pattern: a matched storage becomes a full member of the group, exactly as a literal entry, and `u*` (§5.3 C6) divides by every member's `c_s` — so a pattern that captures an ISO-only or otherwise unusable LUN skews the balance target even though (C2) keeps any disk from ever landing there. Eligibility is unchanged and still per storage; safety never depends on the pattern being precise, but the quality of the balance does.

---

12. Implementation phases

Each phase is independently testable and useful on its own.

#	Phase	Done when
1	`config.py`, `metrics.py`, `pve-storage-drs verify-metrics`	Real metric/label names confirmed against the live Prometheus; per-disk load printed

#	Phase	Done when
2	`pve.py`, `topology.py`	`pve-storage-drs show-load` prints every storage with its disks (all buses), sizes, loads and reserve status; pinned disks flagged with their reason; `pve-storage-drs verify-storages` reports saferemove, implied wipe times and the expansion of every `/…/` storage pattern (§11.4)
3	`loadmodel.py` + gates	Correct act/no-act decision per group, with the reasoning shown
4	`heuristic.py` + `schedule.py`	End-to-end plan in `dry-run`, ordered and transient-feasible; reproduces `expected_order` and `expected_final_reserve` in the §14 fixture
5	`payback.py`	Plans rejected/trimmed on cost grounds, arithmetic shown
6	`optimize.py` (MILP)	Matches or beats the heuristic on the §14 fixture; CP-SAT and CBC agree on every β case, and the §5.5 coefficient assertions pass
7	`execute.py`	`confirm` mode against a lab cluster, including a VM locked mid-run and a source storage with `saferemove` on — the run must wait, not fail
8	`auto` mode + time windows	Unattended operation
9	`forecast.py` beyond p95	Seasonal-naive validated by backtest

Phase 4 before phase 6 is deliberate: a working heuristic makes the MILP verifiable, and it is the production fallback for large groups. Do not start with the solver.

---

13. Failure modes and safety

Hazard	Handling
Metric gap / disk below `min_coverage`	Use last known load from state; flag in output; never treat as zero
Counter reset (VM reboot, node migration)	Handled by `rate()`; `sum by (vmid, device)` collapses node labels
VM live-migrated between nodes mid-plan	Re-fetch node before each move (§9.2); mismatch → abort move, re-plan
Disk has an existing snapshot chain	Pinned, load and bytes still counted, reported at WARN every run with the pinned load/bytes per VM (§3.7). `move_disk delete=1` is rejected by PVE on such volumes and would not carry the snapshots anyway
Snapshot created between planning and execution	Re-checked immediately before every move (§9.2 step 5); the move is dropped and the plan re-planned
VM has `efidisk0` / `tpmstate0`	Ordinary movable disks on PVE 9.2 (verified on a live cluster). Small, so γ/payback are negligible while β charges a full move — tune β and κ together. Do not assume `drive-mirror` semantics for `tpmstate0`; §8.1's both-storages invariant holds either way (§3.6)
VM has disks on `ide`/`sata`/`virtio`, not just `scsi`	Full bus regex in §3.5; enumerating only `scsi*` silently mis-accounts capacity
`unused{N}` volumes	Movable with $\ell\_d = 0$, so the solver relocates them only to repair a reserve violation — the intended policy

Hazard	Handling
VM is `locked` (backup, snapshot, migrate, …)	Pinned at planning time, waited for at execution time up to `execution.locks.wait_timeout`; the lock value set is treated as open-ended and never whitelisted (§9.3)
Source space not reclaimed after a successful move	`saferemove` zeroes the old volume at ~10 MiB/s; the move stays in the `draining` state and keeps charging the source until the volume is observed gone (§8.2, §9.3)
Next move blocked by the previous move's wipe	Completion requires task OK **and** source volume absent **and** lock clear; `cooldown_per_storage` validated against the wipe time (§9.3)
Thin provisioning	`assume_thick_provisioning: false` uses allocated size from `/content`; note allocation can *grow* during a move
Foreign volumes on a storage	Counted via `count_foreign_volumes`; otherwise the reserve silently overstates free space
Orphaned target volume after a failure	Detected and reported, never auto-deleted (§9.3)
Storage already violating the reserve	Soft slack `r_s` keeps the model feasible; violation bypasses gates and is scheduled first
Two DRS instances running	Advisory lock in `state.json` plus a startup scan for in-flight `move_disk` UPIDs owned by the DRS user. The lock is node-local; only the UPID scan crosses the cluster (§11)
Config edited mid-run, cluster-wide	The config is read once at startup and never re-read; the resolved path and its SHA-256 are logged, so a plan can be traced to the exact file that produced it
Storage `/…/` pattern matches nothing in the cluster	Hard error before planning, exactly like a literal id that does not exist: the likelier cause is a typo, and the alternative is a silently shrunken group (§11.4)
Cluster storage set changes after the config is written	Patterns are re-expanded against the live inventory every run, so a new LUN joins its group with no config edit; the expansion is logged and shown by `verify-storages` (§11.4)
Solver infeasible or timing out	Fall back to the heuristic; never emit a partial/unvalidated assignment
`bwlimit` misunderstood	It is **KiB/s** in the API; config is bytes/s and must be converted
PVE Dynamic Load Balancer moves a VM mid-plan	Node re-fetched before every move (§9.2); mismatch triggers a bounded re-plan
Engine crashes mid-move	`inflight_upids` in `state.json` + startup scan; the PVE task continues regardless
Concurrent moves onto one storage	Generalized transient invariant over the in-flight set (§8.1)
Config knob with no effect	§11.1 validation; every knob maps to exactly one formula (§15)

Overarching rule: **the reserve constraint is never traded against balance.** Stated precisely: with the lexicographic solve of (C5) this is exact — the reserve shortfall is minimized in a prior stage that the balance objective cannot influence at any weight. With the single-stage big-M alternative it is *effectively* rather than *provably* hard, because `P` is a calibrated constant; §5.3 gives the bound `P` must clear. Either way `Σ r_s > 0` means physically impossible, not merely unattractive, and (C5) is enforced before, during and after every move.

---

14. Worked example

A complete, self-consistent fixture. Implementations must reproduce these numbers exactly.

The machine-readable form lives in **`tests/fixtures/fc-tier1.yaml`** (input) and **`tests/fixtures/fc-tier1.expected.json`** (expected derivations for every β in the input's `beta_values` sweep, the execution order with its transient checks, the post-plan reserve state, and both payback calculations). Assert against those files in CI rather than transcribing the tables below.

The expected file is **generated, not written**: `tests/fixtures/generate_expected.py` enumerates all 3^6

= 729 assignments per β, so the recorded optimum is proven rather than hand-worked, and derives the order with the §8.2 rule and the §8.1 transient predicate. Run it with `--check` in CI to assert the committed file is current; that check is also the regression test for §5.5's coefficient scaling, since a scaling bug shows up as a different optimum.

This example cannot test everything, and one gap is worth naming. In `fc-tier1` every reserve-violating assignment is *also* worse on balance, so the lexicographic solve and the single-stage big-M solve agree here at **any** `P ≥ 0` — the fixture simply never exercises the distinction the two options of §5.3 exist to make. `tests/fixtures/reserve-tradeoff.yaml` is the companion fixture that does; see §14.6.

14.1 Input

Group `fc-tier1`, three storages of 8.0 TiB each, all `capability_weight = 1.0`, `f = 2.0`, no foreign volumes. `config/drs.example.yaml` uses the same group and storage names with the same weights, so the example config and this fixture agree; the differing-weight feature is illustrated on `fc-tier2` there instead. `ℓ_d` is in average in-flight I/O requests (§4), *not* normalized to sum to one — which is what makes the `ω = 1.0` per mirror endpoint in §14.5 commensurable with it.

Disk	VM	`z_d` (TiB)	`ℓ_d`	On
`101:scsi0`	101	2.0	3.0	san-a
`101:scsi1`	101	1.0	1.0	san-a
`102:scsi0`	102	1.5	2.5	san-a
`103:scsi0`	103	0.5	0.4	san-b
`104:scsi0`	104	1.0	0.3	san-b
`105:scsi0`	105	0.5	0.2	san-c

`Σℓ = 7.4`, so `u* = 7.4 / 3 = 2.4667`.

14.2 Initial state

Storage	`L_s`	used	`Z_s`	`used + f·Z_s`	vs `C_s`
san-a	6.50	4.5	2.0	**8.5**	8.0 → **violates by 0.5 TiB**
san-b	0.70	1.5	1.0	3.5	8.0 ✓
san-c	0.20	0.5	0.5	1.5	8.0 ✓

- Spread `(6.50 − 0.20)/2.4667 = 2.554` → **255%**, far above the 20% gate.
- `E_before = |6.50−2.4667| + |0.70−2.4667| + |0.20−2.4667| = 4.0333 + 1.7667 + 2.2667 = 8.0667`
- san-a already breaches the snapshot reserve. This is why (C5) carries slack `r_s` rather than being hard — a hard constraint would report *infeasible* here and refuse to help.

14.3 Solution

With default weights (`α=1.0, β=0.25, γ=0.05/TiB, κ=0.5`) the optimum is **three** moves:

1. `102:scsi0` san-a → san-c
2. `101:scsi1` san-a → san-b
3. `105:scsi0` san-c → san-b

Storage	`L_s`	used	`Z_s`	`used + f·Z_s`	✓
san-a	3.00	2.0	2.0	6.0	✓
san-b	1.90	3.0	1.0	5.0	✓
san-c	2.50	1.5	1.5	4.5	✓

`E_after = 0.5333 + 0.5667 + 0.0333 = 1.1333`, spread **44.6%**. The reserve violation is repaired.

The β knob, demonstrated. The third move improves imbalance by only `1.5333 − 1.1333 = 0.400`. Its objective contribution is:

```
α·ΔE + β·1 + γ·0.5 TiB  =  −0.400 + 0.250 + 0.025  =  −0.125   → accepted at β=0.25
                        =  −0.400 + 0.500 + 0.025  =  +0.125   → rejected at β=0.50
```

At `beta_move_count: 0.5` the solver returns the **two-move** plan instead, ending at `(3.00, 1.70, 2.70)`, `E = 1.5333`, spread 53%. This is exactly the "minimal number of migrations" trade-off made explicit and tunable; both plans are correct, and β chooses.

The affinity trade-off, demonstrated. Both plans split VM 101 (`scsi0` on san-a, `scsi1` on san-b), incurring `κ = 0.5`. Keeping VM 101 together forces san-a to `L = 4.0` and the best reachable `E` becomes `3.133`. Comparing: `3.133 + 0` (together) vs `1.1333 + 0.5` (split) = `1.633`. Splitting wins by 1.5, so high I/O legitimately overrides the affinity preference — precisely the intended behaviour.

14.4 Ordering

`102:scsi0` is scheduled first: it alone repairs san-a's reserve violation (`4.5 → 3.0` used, so `3.0 + 4.0 = 7.0 ≤ 8.0`), and it also has the largest imbalance reduction. Transient checks:

```
move 1 → san-c:  used 0.5 + 1.5 = 2.0,  max(Z_c, 1.5) = 1.5,  2.0 + 3.0 = 5.0 ≤ 8.0  ✓
move 2 → san-b:  used 1.5 + 1.0 = 2.5,  max(Z_b, 1.0) = 1.0,  2.5 + 2.0 = 4.5 ≤ 8.0  ✓
move 3 → san-b:  used 2.5 + 0.5 = 3.0,  max(Z_b, 0.5) = 1.0,  3.0 + 2.0 = 5.0 ≤ 8.0  ✓
```

14.5 Payback

At `bwlimit = 200 MiB/s`, `ω_src = ω_dst = 1.0`, `H = 7d = 604800 s`, `λ = 10`, and **`saferemove` off on all three storages** so `duration_wipe_d = 0` (§7.1), for the two-move plan:

That last assumption is not a detail of the prose — the config default is `migration.account_saferemove_wipe: true`, and with a wipe at the PVE default of 10 MiB/s the 1.5 TiB move below would carry ~44 h of zeroing on top of its 2.2 h mirror and the arithmetic would look nothing like this. So the fixture states it in the data rather than leaving it to be inferred: `saferemove: false` on each storage, `account_saferemove_wipe: false` in the `migration` block, and an `assumptions` object echoed into the expected file. The per-move records carry `duration_mirror_seconds` and `duration_wipe_seconds` separately, so a run with wiping enabled is a different number in a named field rather than a silent discrepancy.

Move	Size	Duration	`cost = 2 × duration`
`102:scsi0`	1.5 TiB	7 864 s (2.18 h)	15 729 load·s
`101:scsi1`	1.0 TiB	5 243 s (1.46 h)	10 486 load·s
		Σ	**26 214 load·s**

```
benefit = ΔE · H = 6.5333 × 604 800 = 3 951 360 load·s
ratio   = 3 951 360 / 26 214 = 150.7   ≥ λ = 10   → ACCEPT
```

A move that fails payback. Consider instead a 4.0 TiB archive disk with `ℓ = 0.1` whose relocation would improve `E` by only 0.05:

```
cost    = 2 × (4.0 TiB / 200 MiB/s) = 2 × 20 972 = 41 943 load·s
benefit = 0.05 × 604 800 = 30 240 load·s
ratio   = 0.72   <  λ = 10   → REJECT
```

The migration would generate more I/O than it saves within the horizon. This is the requirement that "migrating a very large disk might generate more traffic than we are trying to save", enforced numerically.

14.6 Companion fixture: when the reserve and the balance objective disagree

`tests/fixtures/reserve-tradeoff.yaml` is a second, deliberately awkward group, and its only job is to separate the two solve paths of §5.3.

Two storages: `roomy` (20 TiB, empty) and `cramped` (5 TiB, of which 3 TiB is already U^{sext} — volumes DRS does not manage). `f = 2.0`. Two disks, both 1.0 TiB, both ℓ`_d = 5.0`, both on `roomy`, belonging to different VMs. So `u* = 5.0`, and:

Assignment	Σ r_s	E	Non-reserve objective at β = 0.25
both on `roomy` (current)	**0**	10.0	10.0
one moved to `cramped`	1.0 TiB	**0.0**	0.30
both moved to `cramped`	2.0 TiB	10.0	10.60

Moving one disk balances the group *perfectly* and costs a 1 TiB reserve breach on `cramped` (`3 + 1 + 2·1 = 6 > 5`). That is the trade the reserve rule exists to forbid, and the three answers are:

- **Lexicographic (the default).** Stage 1 finds `min Σ r_s = 0`, stage 2 optimises within that set — so the plan is *no moves at all*, and the group stays at `E = 10`. Correct, and it needed no calibration to be correct.
- **Big-M at the configured `P = 1000`.** Same answer: `0.30 + 1000 > 10.0`.
- **Big-M at `P = 5`.** `0.30 + 5 = 5.30 < 10.0`, so it moves the disk and breaches the reserve for balance. The exact flip point, recorded in the expected file as `big_m_agreement_threshold_p`, is **`P = 9.7`**: below it big-M is wrong, above it big-M is right. Note how small that number is — nothing about `P = 5` looks obviously wrong to an operator, which is the whole argument for computing `P` rather than configuring it.

The build-time bound of §5.3 gives $P_{min} = 21.6 \cdot 2^{20} \approx 2.26 \times 10^7$ for this group, four orders above the 9.7 actually needed. That is the bound doing its job: it is deliberately worst-case (it refuses to trade even one mebibyte), and being conservative in the safe direction costs nothing.

One further check the fixture records: the `P = 5` plan is not merely undesirable, it is **unschedulable**. Its single move fails the §8.1 transient predicate on `cramped`, so §8.2 reports a deadlock rather than emitting it. The two safety mechanisms are independent, and the fixture asserts that both fire.

Disk `201:scsi0` and `202:scsi0` are interchangeable here; the recorded move names `202:scsi0` because that is how the enumerator breaks the tie. An implementation may pick either — assert on the move *count* and the resulting slack, not on the disk identity.

---

15. Requirements traceability

Original requirement	Where addressed
Proxmox API via user/password	§3.5
Per-disk metrics for all running VMs	§3.1, §3.4 (blockstat via Prometheus, not RRD — see §3.2)
Equalize I/O across storages in a group	§4, §5.3 (C6), §5.4
VMs confined to their storage group	§5.1, §5.3 (C1, C2)
Restrict to shared storages	§5.3 (C2)
2× largest disk free for snapshots	§5.3 (C5)
...including *during* migrations	§8.1 transient invariant
Reserve factor configurable	`snapshot_reserve.factor`, per-storage override
Minimal number of migrations	§5.4 β term; demonstrated §14.3
Min % changed traffic before acting (10%)	§6 drift gate
% I/O difference across the group	§6 imbalance gate

Original requirement	Where addressed
Keep a VM's disks together, unless space/IO forces otherwise	§5.3 (C3), §5.4 κ; demonstrated §14.3
Mathematical optimization formulation	§5
Migration order planned	§8
Automatic or manual confirmation	§9.1 (`dry-run` / `confirm` / `auto`)
Forecasting (Holt-Winters or other)	§10
Configurable decision timeframe, default 24h	`window.lookback`; §3.4
Migration load counted against the benefit	§7; demonstrated §14.5

15.1 Config knob → formula

Every knob must appear in exactly one formula. A knob with no formula is dead configuration and a bug waiting to happen; this table is the audit.

Knob	Where it acts
`load_weights.iotime/ops/bytes`	§4, ℓ_d
`load_weights.read_factor/write_factor`	§4, `raw_X(d)`, applied engine-side before normalization
`window.lookback`	§3.4 reduction range
`window.quantile` / `upper_quantile`	§10.1, point estimate (the actual decision statistic) vs. the bound (§7.3 saturation guard only — see the "As built" note in §10.1)
`window.min_coverage`	§3.4, disk data rejection
`groups[].storages[].id` in pattern form (`/…/`)	§11.4 expansion into group membership; the entry's options apply to every matched storage
`groups[].storages[].capability_weight`	§4, `u_s = L_s / c_s`
`snapshot_reserve.factor`	§5.3 (C5), $R\_s \geq f\_s \cdot Z\_s$
`snapshot_reserve.min_free_bytes`	§5.3 (C5), $R\_s \geq \text{min\_free\_bytes\_s}$
`snapshot_reserve.count_foreign_volumes`	§5.1.1, U^{sext}
`gates.drift_threshold`	§6 drift gate
`gates.imbalance_threshold`	§6 imbalance gate
`gates.cooldown_per_disk/storage`	§5.3 (C2) pinning, §8.1 `concurrency_ok`
`migration.bwlimit_bytes_per_sec`	§7.1 `duration_d`; converted to KiB/s at the API call
`migration.source/target_load_weight`	§7.1 ω_src, ω_dst
`migration.payback_horizon` / `payback_ratio`	§7.2, §7.3 acceptance test
`migration.max_single_move_duration`	§7.3 hard per-move rule; compared against `duration_d` *including* the wipe
`migration.account_saferemove_wipe`	§7.1 `duration_wipe_d`
`migration.wipe_load_weight`	§7.1 ω_wipe in `cost_d`; §7.3 $\omega\_role(m,s)$ while `draining`
`execution.locks.*`	§9.3 lock wait loop; §5.3 (C2) planning-time pin
`execution.source_release.*`	§9.3 completion criterion; §8.2 `draining` state
`exclude.include_unused_disks`	§3.6 membership of `D`
`exclude.skip_vms_with_snapshots`	§3.7, §5.3 (C2) pinning
`objective.affinity_counts_pinned_disks`	§5.3 (C3) range of `D^mov`
`report.warn_pinned_load_fraction`	§3.7 unreachable-goal warning
`migration.saturation_ceiling`	§7.3 $L\_during(s) \leq \text{saturation\_ceiling} \cdot N\_s$
`groups[].storages[].saturation_load`	§7.3 N_s; guard skipped when unset
`objective.alpha_spread/beta_move_count/gamma_move_bytes_per_tib/kappa_vm_affinity`	§5.4
`objective.reserve_violation_penalty`	§5.3 (C5), *floor* for the single-stage `P` alternative
`metrics.pvestatd_push_interval`	§11.1 `rate_window` validation; §3.3 `verify-metrics`
`objective.spread_metric`	§5.3 (C6), L1 vs min–max
`solver.*`	§5.5
`execution.max_concurrent_migrations/per_storage`	§8.1 generalized invariant, `concurrency_ok`
`execution.max_replans_per_run`	§9.2 re-plan protocol
`execution.time_windows`	§9.1 `auto` mode gating
`exclude.*`	§5.3 (C2) variable fixing
`forecast.model` + `holt_winters.*`	§10.1
`proxmox.read_workers`	§3.5 read-path concurrency